

University of Science
Faculty of Information Technology

EduFlex

Design and Implementation of a Mobile Learning Platform

Student: Vu Thanh Dat
Student ID: 23127346
Supervisor: Dr. Truong Toan Thinh
Submission: October 2026

Project Report

Declaration

I declare that this report describes my own project work. Sources used to explain technologies, standards, and external systems are acknowledged in the references. Project-specific results are based on the source code and verification records in the EduFlex repository.

Student signature: _____

Date: _____

Acknowledgements

I would like to thank my supervisor, lecturers, classmates, and testers for their guidance and feedback during the development of this project. Their comments helped shape both the learning experience and the engineering decisions in this report.

Abstract

EduFlex is a native Android learning platform that connects course discovery, enrolment, lesson study, assessment feedback, and visible progress. XP, streaks, quests, badges, and a leaderboard support continued interaction, while an administration area manages users and learning content. The project investigates how a mobile learning journey can be supported by consistent backend state and observable operations.

The Java/XML client uses Material components and Retrofit to communicate with a Spring Boot API. PostgreSQL is authoritative; Flyway versions the schema and jOOQ expresses explicit SQL. Redis caches reads and tracks refresh sessions. RabbitMQ defers content embedding, with local MiniLM vectors stored through pgvector. Optional Gemini generation uses retrieved course material. Docker Compose supplies the local topology, GitHub Actions automates verification, and Prometheus with Grafana and Alertmanager exposes operational signals.

The principal engineering result is tested protection of learning transitions under retry and concurrency. Transactions, learner row locks, conditional writes, and unique constraints coordinate progress and protected rewards. On 4 October 2026, all 28 backend tests passed, including 23 PostgreSQL integration scenarios and a clean application of 28 migrations. One hundred simultaneous duplicate lesson-completion calls left one progress row and one 20-XP reward. Earlier Android, live HTTP, release build, and monitoring records are distinguished from this fresh backend run.

The report traces these results through architecture, data models, API contracts, worked progression examples, failure analysis, and operational recovery design. Source review also identifies incomplete assessment validation, unequal quiz workflows, client-reported quest activity, order-sensitive course reward eligibility, and broker publication gaps. These boundaries qualify the tested consistency claims. Learning effectiveness, usability, sustained performance, and production recoverability require further experiments; this report does not infer those outcomes from the technology stack alone.

Keywords: mobile learning, Android, Spring Boot, PostgreSQL, concurrency, semantic retrieval, messaging, observability.

Contents

Declaration	i
Acknowledgements	ii
Abstract	iii
List of Abbreviations	xii
1 Introduction	1
1.1 Background	1
1.2 Problem statement	1
1.3 Research and evaluation questions	2
1.4 Objectives	2
1.5 Scope and limitations	2
1.6 Report organization	3
2 Related Work and Design Position	4
2.1 Mobile learning	4
2.2 Gamification	4
2.3 Retrieval-grounded assistance	4
2.4 Project contribution	5
2.5 Comparison with established platforms	5
3 Requirements Analysis	6
3.1 Actors	6
3.2 Functional requirements	6
3.3 Use cases and acceptance conditions	7
3.4 Non-functional requirements	8
3.5 Requirement prioritization	9
4 Technologies and Tools	10
4.1 Native Android	10
4.2 Spring Boot	10

4.3	Persistence	10
4.4	Caching, messaging, and delivery	11
4.5	Technology selection rationale	11
5	System Architecture and Design	13
5.1	Architectural argument	13
5.2	Overview	14
5.3	Backend structure	15
5.4	Android structure	16
5.5	Deployment	16
5.6	Decisions and rejected alternatives	18
5.7	Request and failure boundaries	19
6	Database Design	20
6.1	Core model	20
6.2	Aggregate boundaries and relational cardinality	21
6.3	Ownership and lifecycle	22
6.3.1	Assessment definitions and activity records	23
6.3.2	Reward definitions and daily state	24
6.3.3	Deletion, retention, and reserved fields	24
6.4	Schema evolution	25
6.5	Integrity and connections	26
6.6	Relational and vector data together	27
6.7	Queries, indexes, and connection budget	27
7	Mobile Application Implementation	29
7.1	User experience	29
7.2	Networking and authentication	31
7.2.1	Coordinating concurrent token refresh	31
7.2.2	Transport budgets and device state	32
7.3	Learning and engagement	33
7.4	UI quality and accessibility considerations	33
7.5	Build variants	34
8	Backend and Security Implementation	35
8.1	REST API	35
8.2	API contract groups	36
8.3	Assessment execution and grading boundaries	37
8.4	Activity trust and overlapping contracts	39
8.5	Authentication and authorization	40

8.6	Single-use password recovery	41
8.7	Development checkout and enrolment integrity	42
8.8	Contract errors and authorization threats	43
8.9	Media	45
8.10	Redis cache policy	45
8.10.1	Cache invalidation and consistency boundary	46
9	Reliability, Concurrency, and Messaging	47
9.1	Transactional updates	47
9.1.1	Invariants rather than successful responses	47
9.2	Per-learner serialization	49
9.3	RabbitMQ embeddings	50
9.3.1	Delivery semantics and stale vectors	50
9.3.2	Calendar time and reward semantics	51
9.4	Worked progression examples	51
9.5	Known retry boundary	53
9.6	Failure analysis	53
10	AI-Assisted Learning	55
10.1	Retrieval	55
10.1.1	Catalogue search and course question retrieval	55
10.1.2	Scoring and context construction	56
10.2	Grounded generation and fallback	56
10.3	Responsible boundaries	57
10.4	Quality evaluation approach	57
11	DevOps, Deployment, and Monitoring	59
11.1	Containerization and configuration	59
11.2	Continuous integration	59
11.3	Metrics, dashboards, and alerts	59
11.4	Operational security	61
11.4.1	What custom metrics do and do not establish	61
11.4.2	Exposure and access	61
11.5	Operational interpretation	61
11.5.1	Measured signals and configured thresholds	62
11.5.2	Operational maturity and deployment limits	63
11.6	Recovery design and state classification	63
11.6.1	Reproducibility and deployment drift	65

12 Testing and Evaluation	67
12.1 Strategy	67
12.2 Backend results	69
12.3 End-to-end and Android results	70
12.4 Build and monitoring results	71
12.5 Evaluation limits	71
12.5.1 Answers to the evaluation questions	71
12.5.2 Threats to validity	72
12.5.3 Source audit findings outside the passing suite	72
12.5.4 Unmeasured product and performance outcomes	74
12.6 Performance claims and missing measurements	74
12.6.1 A repeatable measurement protocol	75
13 Conclusion and Future Work	77
13.1 Conclusion	77
13.2 Future work	77
13.3 Prioritized engineering roadmap	78
13.4 Portfolio summary	79
A Build and Verification Commands	80
B Selected API Groups	81
B.1 Methods, identity, and policy differences	81
B.2 Implementation entry points	83
C Configuration Reference	85
D Diagram and Evidence Index	87
E Submission Checklist	89

List of Figures

5.1	Responsibility boundaries: authoritative, derived, and observed state . . .	14
5.2	System context and external actors	15
5.3	EduFlex runtime architecture	15
5.4	Backend responsibility layers	16
5.5	Android navigation and principal user journeys	16
5.6	Compose service dependencies: arrows follow callers; broker data storage is not mounted	17
6.1	Domain relationship overview; detailed figures specify keys and cardinality	21
6.2	Learning data model with cardinality and natural-key constraints	22
6.3	Assessment definitions and persisted history; dashed note identifies missing audit detail	23
6.4	Reward definitions, aggregate totals, and per-learner earned relationships .	24
7.1	End-to-end learner course journey	30
7.2	Implemented in-process coordination for concurrent expired-token requests	32
7.3	Screen-state acceptance model for loaded data; device coverage remains partial	34
8.1	API groups and authorization boundaries	35
8.2	Existing quiz workflows differ in scope, persistence, and rewards; grading gaps remain	38
8.3	Login and access-token refresh sequence	41
8.4	Implemented recovery lifecycle when SMTP is configured; not one dis- tributed transaction	42
8.5	Retry-safe development checkout; repeated requests converge on one suc- cessful record	43
8.6	Cache-aside read and invalidation flow	45
9.1	Successful duplicate completion ordering for one learner under a row lock .	48
9.2	Learning-progress state and retry behavior	49
9.3	Content-completion transaction; multiple choice has additional attempt and quest writes	49

9.4	Post-commit content embedding sequence	50
10.1	Grounded course-question flow	57
11.1	Continuous integration order	59
11.2	Metrics, visualization, and alerting flow	60
11.3	Provisioned Grafana Service Overview running against the local stack . . .	60
11.4	Runbook-based investigation and recovery decision model	63
11.5	Proposed isolated restoration drill; backup automation and replay are not implemented	65

List of Tables

3.1	Primary actors	6
3.2	Representative use cases and observable completion conditions	7
3.3	Non-functional requirements	8
4.1	Selected declared versions in the inspected project	11
4.2	Technology choices and project trade-offs	12
5.1	Architectural decisions and their practical consequences	18
6.1	Principal entity groups	20
6.2	Data lifecycle boundaries in the inspected implementation	25
6.3	Selected integrity rules and their purpose	26
7.1	Implemented learner screens and required visible states	30
8.1	Representative API contracts	36
8.2	Implemented assessment behavior by submission type	38
8.3	Concrete threats, implemented controls, and residual limits	44
8.4	Configured Redis cache lifetimes	45
9.1	Source-derived ordering examples for A, B, and quiz child Q	52
9.2	Reward rules and their actual qualifying boundaries	53
9.3	Reliability mechanisms and remaining boundaries	54
11.1	Selected implemented alert conditions	62
11.2	Recovery requirements by stored state	64
12.1	Evidence mapped to quality goals	68
12.2	Claim-to-evidence traceability	68
12.3	Selected verified scenarios	69
12.4	Evaluation questions answered within the available evidence	71
12.5	Uncovered implementation gaps and required regression evidence	72
12.6	Required performance experiment design	75
13.1	Future increments and required proof	78

B.2	Selected route access and contract characteristics	82
B.3	Source entry points for report claims	84

List of Abbreviations

API	Application Programming Interface
CI/CD	Continuous Integration and Continuous Delivery
DTO	Data Transfer Object
FCM	Firebase Cloud Messaging
JWT	JSON Web Token
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SQL	Structured Query Language
TTL	Time To Live
UI/UX	User Interface and User Experience
XP	Experience Points

Chapter 1

Introduction

1.1 Background

Mobile devices make learning material available beyond the classroom, but availability alone does not guarantee effective study. Learners also need a clear path through content, immediate feedback, visible progress, and reasons to return. Mobile learning systems therefore combine content delivery with interaction, assessment, reminders, and progress tracking. Android was selected because it supports a broad range of devices and mature libraries for navigation, background work, networking, and accessibility [7]. A meta-analysis of 110 experimental and quasi-experimental studies reported a moderate average effect for mobile-integrated learning, while showing that outcomes depend on instructional design and context [26]. EduFlex therefore treats the phone as a delivery and interaction channel rather than evidence of educational effectiveness by itself.

EduFlex connects these learner needs to a backend designed for data correctness and operational visibility. The project is also an engineering portfolio: it demonstrates an end-to-end system rather than an isolated interface or collection of endpoints.

1.2 Problem statement

Many prototype learning applications focus on screen design but do not address problems that appear when an action is retried, several requests arrive together, cached data becomes stale, or a dependency fails. One lesson completion can affect progress, XP, quests, course completion, and badges. If these changes are independent, a duplicated request may award a reward twice or leave partially updated data.

The project addresses two connected problems: providing a usable mobile experience for discovering and completing learning content, and preserving trustworthy learning state while supporting caching, asynchronous work, AI assistance, automated testing, and monitoring.

1.3 Research and evaluation questions

The implementation is evaluated through four bounded questions:

1. Can a learner complete the principal account-to-course journey through the Android client and REST API?
2. Do duplicate and concurrent learning commands preserve one authoritative progress transition and one protected reward?
3. Can asynchronous embedding and cache acceleration remain outside the authoritative transaction while failures stay observable?
4. Can a developer reproduce the main build, integration, and monitoring checks from versioned commands and infrastructure definitions?

The report answers these questions with repository inspection and recorded verification. It does not claim that EduFlex improves grades, motivation, or retention because no controlled learner study was conducted.

1.4 Objectives

The main objective is to implement a functional Android learning application backed by a secure and observable service. The specific objectives are:

- provide account, course, lesson, quiz, review, payment-record, and gamification flows;
- present consistent screens with loading, empty, validation, and failure states;
- expose a documented REST API with authentication and role-based access;
- store learning state in a versioned relational schema;
- make progress and rewards safe under retry and concurrency;
- move repeated reads and embedding work into bounded cache and background paths;
- ground optional AI output in stored course material;
- automate build and integration verification; and
- expose application and infrastructure health through dashboards and alerts.

1.5 Scope and limitations

The scope includes a native Android learner application, an Android administration area, Spring Boot, PostgreSQL/pgvector, Redis, RabbitMQ, Docker Compose, tests, GitHub Actions, and a local monitoring stack. Firebase push, Supabase media storage, and Gemini generation are optional integrations. The core development build works without their credentials.

The checkout screen records development payment simulations and does not settle real payments. Recovery requires configured SMTP; without it the application reports that recovery is unavailable. The project does not publish a signed application to Google Play. CI release outputs are unsigned inspection artifacts. Centralized logs, distributed tracing, external alert delivery, and automated database backup are outside the current deployment.

1.6 Report organization

The report moves from requirements and relevant research to implementation and evaluation. It first explains the technology and architectural decisions, then follows data, mobile, API, reliability, AI, and operational concerns. The final chapters map claims to recorded evidence, state evaluation limits, and identify future work.

The report uses three distinct evidence levels. *Implemented* means that a behavior is present in the inspected source. *Verified* means that a named test or recorded experiment exercised it; the evaluation chapter states the date and scope. *Proposed* means that a design, acceptance criterion, or experiment is future work. This distinction applies to diagrams as well as prose. In particular, an acceptance-state diagram or recovery plan must not be read as a completed device test or an executed disaster-recovery drill.

Chapter 2

Related Work and Design Position

2.1 Mobile learning

Sung, Chang, and Liu synthesized 110 studies and found a moderate overall effect for integrating mobile devices into teaching and learning [26]. Their analysis also identified variation across instructional methods, intervention duration, and implementation settings. This result supports mobile access as a useful design opportunity, but it does not justify assuming that any mobile application improves learning. EduFlex consequently focuses on observable interaction qualities—clear progress, feedback, retry behavior, and access to ordered material—while reserving pedagogical effectiveness for a later learner study.

2.2 Gamification

Hamari, Koivisto, and Sarsa found positive effects in many reviewed empirical studies, while emphasizing that outcomes depended on the users and setting [10]. EduFlex uses XP, levels, streaks, quests, badges, and a leaderboard to make activity and progress visible. Their presence does not establish improved motivation. The current tests evaluate reward transitions. A learner study would also need to examine continued use, perceived usefulness, and competitive pressure.

2.3 Retrieval-grounded assistance

Retrieval-augmented generation combines a generator with an explicit retrievable knowledge source, improving provenance and allowing knowledge to be updated outside model parameters [13]. EduFlex applies the same broad pattern at project scale: it restricts retrieval to one course, ranks lesson material through pgvector, and returns source lessons with the answer. This is an engineering safeguard rather than proof of factual correctness.

Retrieval relevance and answer groundedness require a labelled evaluation set that the present project does not yet contain.

2.4 Project contribution

EduFlex’s contribution is the integration of a native learning client with a backend that treats progress integrity as a first-class requirement. Its strongest evaluated contribution is the transaction boundary joining lesson progress, course completion, XP, quests, and badges under duplicate and concurrent requests. Caching, asynchronous embedding, and monitoring are arranged around that boundary so operational features do not become the source of truth. The project is a full-stack engineering case study, not a controlled study of educational outcomes.

2.5 Comparison with established platforms

Google Classroom’s mobile application centres on joining classes, assignments, submission, communication, and limited offline access [6]. Moodle exposes a much broader learning-management feature set through its web and mobile clients, including course lists, activities, badges, and offline content, with some capabilities differing between clients [14]. EduFlex is not evaluated as a replacement for either platform. Its narrower contribution is an inspectable reference implementation in which retry-safe progress and reward updates, local semantic retrieval, and operational telemetry are evaluated together. It lacks the mature collaboration, institutional administration, plugin ecosystem, and offline learning support of established systems.

Chapter 3

Requirements Analysis

3.1 Actors

The learner needs a simple path from discovery to completion. The administrator maintains users and content. An operator runs the system and observes its health. External systems optionally provide media storage, remote notifications, and generated text.

Table 3.1: Primary actors

Actor	Responsibilities and goals
Learner	Authenticate, discover courses, enrol, study, complete quizzes, track progress, earn rewards, and review material.
Administrator	Manage users, courses, lessons, and questions through protected operations.
Operator	Configure, deploy, observe, diagnose, and maintain the platform.
External provider	Supply optional storage, notification, or generation services.

3.2 Functional requirements

The account module shall support registration, login, owner-checked profile update, single-use password recovery when SMTP delivery is configured, token refresh, and logout. The catalogue shall support listing, categories, search, details, reviews, and learner enrolments. Lessons may contain study content or multiple-choice and fill-in-the-blank quizzes. Progress shall be saved and reflected in continue-learning and course completion.

Gamification shall expose XP, levels, daily check-in, streaks, quests, badges, and a leaderboard. Course assistance shall summarize stored material and answer questions

using retrieved context. Administration shall restrict content and user management to an administrator role.

3.3 Use cases and acceptance conditions

Table 3.2: Representative use cases and observable completion conditions

Use case	Main flow	Acceptance condition
Authenticate	Register or log in, then establish a session	Valid credentials open the correct role area; invalid credentials do not.
Discover	Browse categories or search for a course	Results, empty state, and request failure are visibly distinct.
Enrol	Select a free course or run the development payment simulation	One learner-course enrolment exists; paid courses reject the free route.
Study	Open ordered content and mark a lesson complete	Progress changes once and remains stable under a retry.
Complete quiz	Submit multiple-choice or fill-blank answers	Result is returned and protected rewards are awarded only when eligible.
Track motivation	Open statistics, quests, badges, or leaderboard	Current backend state is displayed with a recoverable error state.
Ask course	Request a summary or ask a course question	Response uses course scope and provides lesson sources or fallback guidance.
Administer	Create, update, or remove learning content	A learner is rejected; an administrator can perform the valid operation.
Operate	Start services and inspect health and metrics	Required services become healthy and monitoring targets report up.

These conditions connect product language to evidence. A feature is not considered complete merely because a screen or endpoint exists; its success, empty, denied, duplicate,

and failure behavior must also be understandable.

3.4 Non-functional requirements

Table 3.3: Non-functional requirements

Quality	Observable criterion	Status
Usability	Core data screens expose loading, empty, validation, failure, and retry states.	Partially verified by source inspection; no user study.
Security	Cross-user and learner-to-admin mutations return 403; reset codes expire and are single-use.	Verified for selected routes, not a penetration test.
Consistency	Duplicate and concurrent protected transitions leave one database reward.	Verified by PostgreSQL tests.
Performance	Cache hit ratio, request percentiles, queue delay, and saturation are measured.	Partial telemetry; no controlled benchmark or explicit queue-age metric.
Maintainability	A clean database applies every migration and the documented commands compile.	Fresh backend schema check; Android build evidence is historical.
Testability	Critical rules run against PostgreSQL, including rollback and concurrency.	Verified.
Observability	API loss fires and resolves an alert; dependency targets are separately visible.	Historical local Compose drill.
Portability	Clean Android builds work without Firebase and Compose starts core dependencies.	Historical verification for the recorded environment.

3.5 Requirement prioritization

Account security, course access, progress integrity, and build reproducibility are mandatory because other features depend on them. Caching, message processing, and monitoring improve performance and operations once the authoritative learning path is correct. Optional external integrations add value but must not prevent a clean developer build.

This priority influenced implementation order. PostgreSQL transactions and ownership checks guard the core path. Redis and RabbitMQ surround that path without becoming the authority for progress. Gemini, Firebase, and Supabase are configured at the edge and can be absent in local or CI environments.

Chapter 4

Technologies and Tools

4.1 Native Android

The client uses Java 17, XML layouts, minimum SDK 24, and target SDK 35. Material Components, AppCompatActivity, and ConstraintLayout provide consistent UI behavior. Navigation Component represents destinations and arguments in XML. View Binding gives compile-time access to views.

Retrofit maps REST endpoints to Java interfaces, Gson maps JSON models, and OkHttp handles transport and debug logging [25]. Glide loads images. WorkManager schedules persistent local reminders [8]. Firebase Messaging is optional and activated only by an untracked configuration file.

4.2 Spring Boot

Spring Boot 3.4.2 provides dependency injection, web routing, validation, security, data access, health endpoints, and external configuration [3]. Controllers translate HTTP, services contain business rules, and repositories isolate persistence. Springdoc provides OpenAPI and Swagger UI.

4.3 Persistence

PostgreSQL 16 supplies transactions, constraints, row locking, indexing, and vector extension support [17]. Flyway applies 28 versioned migrations [20]. Hibernate validates mappings without creating the schema. jOOQ supports operations where locking and conditional SQL must be explicit [4]. pgvector stores 384-dimensional MiniLM vectors beside relational course data [11].

4.4 Caching, messaging, and delivery

Redis stores caches and revocable refresh-token records [21]. RabbitMQ and Spring AMQP move content embedding away from request processing [2]. Docker Compose describes the runtime environment. Gradle and Maven wrappers provide repeatable builds. JUnit and Testcontainers exercise PostgreSQL/pgvector integration [1]. GitHub Actions coordinates backend, monitoring, and Android verification.

Table 4.1: Selected declared versions in the inspected project

Area	Declared version	Source of truth
Android platform	Compile/target SDK 35; minimum 24	Android module build file
Android language	Java 17	Compile options
Material components	1.12.0	Android dependencies
Retrofit / OkHttp	2.11.0 / 4.12.0	Android dependencies
WorkManager	2.10.1	Android dependencies
Spring Boot	3.4.2; Java target 17	Maven parent and properties
Backend verification	JDK 21; Docker Engine 29.8.1	Recorded 4 October run
Database family	PostgreSQL 16 with pgvector	Compose image and integration fixture
Monitoring	Prometheus 3.5.0; Grafana 12.1.0	Monitoring Compose file
Alertmanager	0.28.1	Monitoring Compose file

These are project-declared versions and one recorded verification environment, not a recommendation to use the latest upstream release. The Java compilation target and the JDK used to run the tests are deliberately listed separately.

4.5 Technology selection rationale

Table 4.2: Technology choices and project trade-offs

Choice	Reason in EduFlex	Trade-off
Native Java/XML	Fits project convention, Android SDK access, and explicit layouts	More manual screen-state coordination than some newer UI stacks.
Spring Boot	Integrated web, validation, security, data, health, and configuration	Framework conventions require clear boundaries to avoid oversized services.
PostgreSQL	Transactions, locks, constraints, indexing, and pgvector in one system	Scaling writes still requires careful query and pool design.
jOOQ plus JPA	JPA handles mappings; jOOQ expresses locks and conditional SQL clearly	Two data-access styles need consistent repository conventions.
Redis	Fast bounded caches and refresh-token revocation	Availability and invalidation become operational concerns.
RabbitMQ	Decouples content writes from embedding work	Delivery durability needs an outbox and dead-letter policy.
Local MiniLM	Embeddings work without sending course text to a generation provider	Model changes require vector migration and reprocessing.
Docker Compose	Repeatable multi-service development and monitoring	It is a single-host development topology, not an orchestration platform.
Metrics stack	Versioned Prometheus rules and Grafana dashboards	Logs and traces remain separate future work.

The selected technologies serve specific responsibilities. PostgreSQL remains the authority, Redis accelerates derived reads, and RabbitMQ transports deferred work. This distinction is central to failure handling: losing a cache entry should cost time, while losing an authoritative progress row would cost correctness.

Chapter 5

System Architecture and Design

5.1 Architectural argument

EduFlex is a modular application deployed as one Spring Boot process, accompanied by specialized data and infrastructure services. It is not a collection of independently deployable business microservices. Course, assessment, enrolment, and reward modules share one relational database and can participate in the same transaction. This is useful at the current scale because completing a lesson must update several related facts without introducing a distributed commit protocol.

The design separates the *authoritative path* from the *derived path*. Authenticated learning commands reach transactional services and PostgreSQL. Catalogue caches and embedding vectors are derived representations: their absence or temporary staleness must not authorize a learner or mint a reward. The monitoring path observes both without becoming a dependency of each application request.

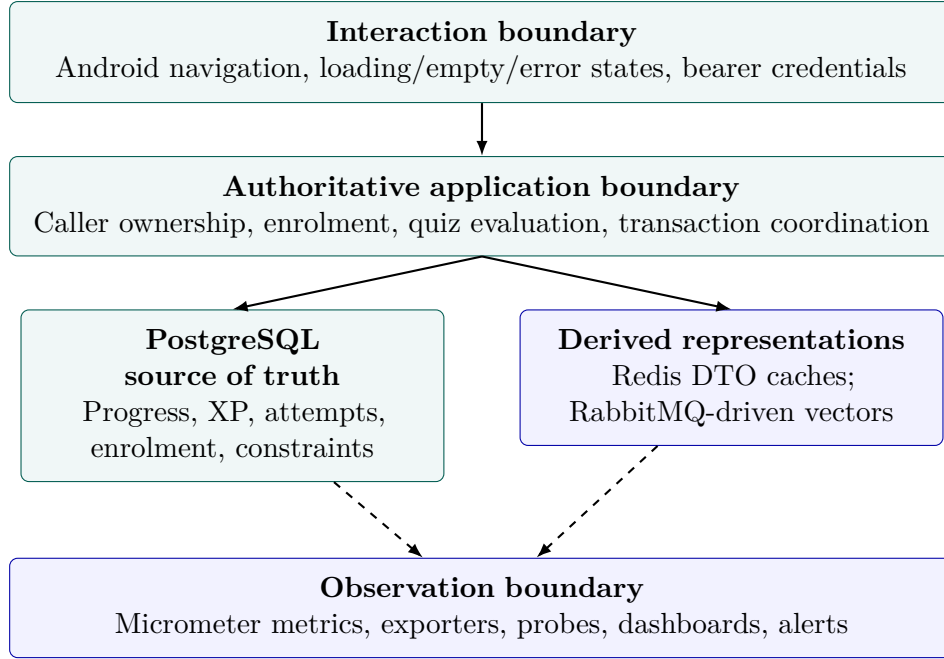


Figure 5.1: Responsibility boundaries: authoritative, derived, and observed state

This separation also defines what a future deployment can scale. Additional API instances could share PostgreSQL and Redis because critical serialization occurs in the database rather than a Java monitor. Embedding consumers could be separated from HTTP serving to isolate CPU-heavy inference. Neither topology has been load-tested here, so those are architectural extension points rather than demonstrated capacity.

5.2 Overview

The Android app sends JSON over HTTP and attaches an access token to authenticated calls. The backend owns authorization, domain rules, storage, caching, messaging, AI orchestration, and metrics.

The architecture is described through several complementary views. The context view in Figure 5.2 identifies users and systems outside EduFlex. The runtime view in Figure 5.3 then shows where requests and data travel. An optional provider can therefore fail without changing which component owns authoritative learning state.

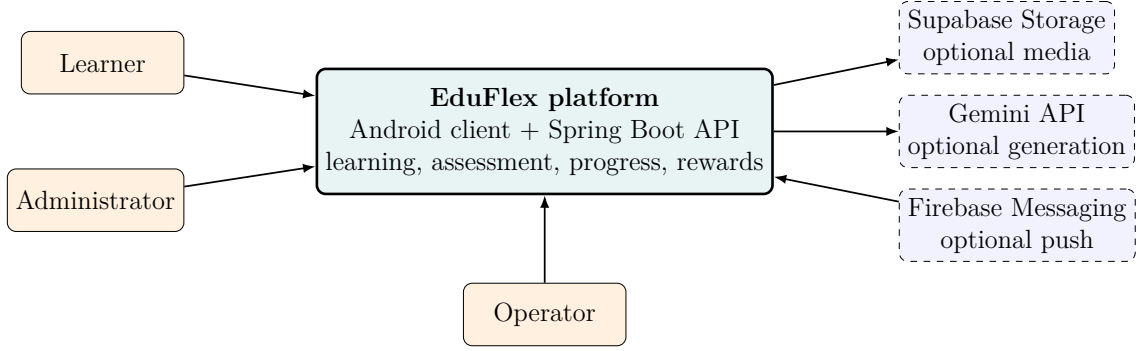


Figure 5.2: System context and external actors

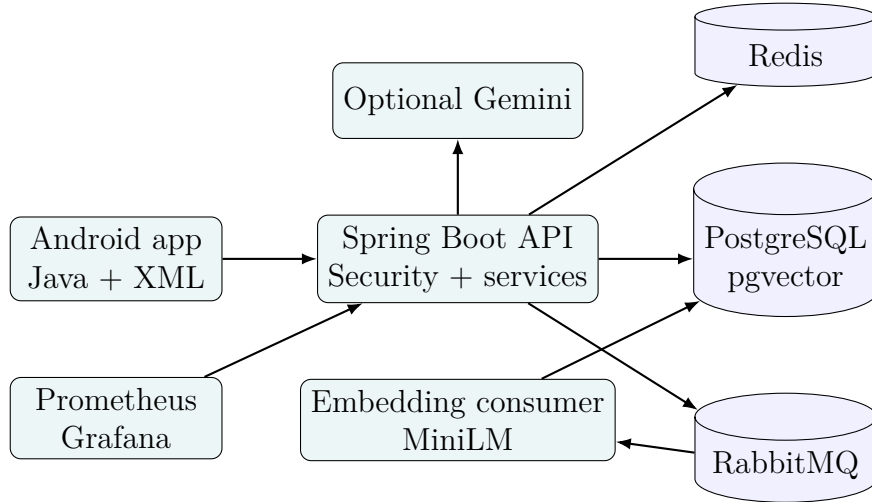


Figure 5.3: EduFlex runtime architecture

5.3 Backend structure

Controllers define transport contracts and delegate to services. DTOs isolate wire payloads from database records. Use-case classes perform validation and coordinate repositories. Repositories contain JPA and jOOQ operations. Configuration supplies security, caches, messaging, clocks, and metrics. This structure keeps business invariants out of the mobile client and HTTP layer.

Figure 5.4 makes the dependency direction explicit. Controllers call application services, and services use repositories or integration clients. Persistence code does not select HTTP responses, while Android does not calculate authoritative rewards. These boundaries allow direct service testing and reduce the effect of transport or database implementation changes.

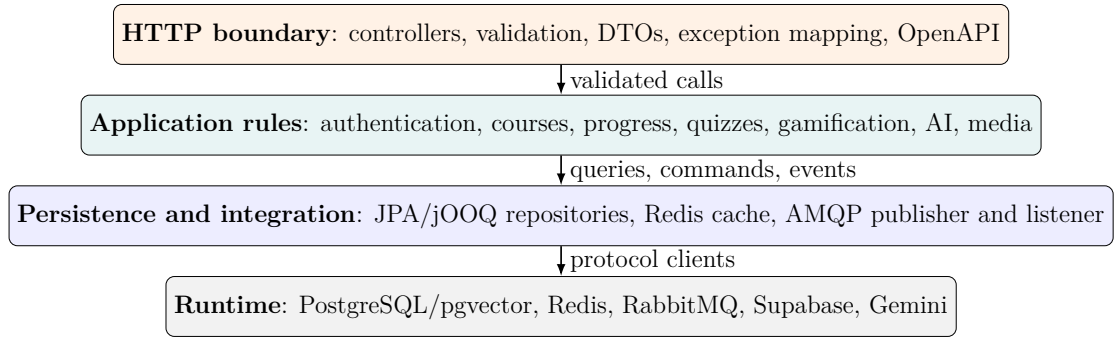


Figure 5.4: Backend responsibility layers

5.4 Android structure

Authentication activities lead to learner or administrator navigation. Learner destinations include Home, Courses, Search, Cart, Profile, Payment, Course Detail, Lesson Study, two quiz modes, Leaderboard, Reviews, Certificate, and AI Summary. Administrator destinations include Users, Courses, Lessons, and Profile. API interfaces, models, session managers, and list adapters support these screens.

Figure 5.5 shows the learner path and the separate administrator branch. Course Detail is a central decision screen that connects discovery to enrolment, study, reviews, and AI support. Stable navigation IDs and explicit arguments keep the course and lesson identity consistent across screens.

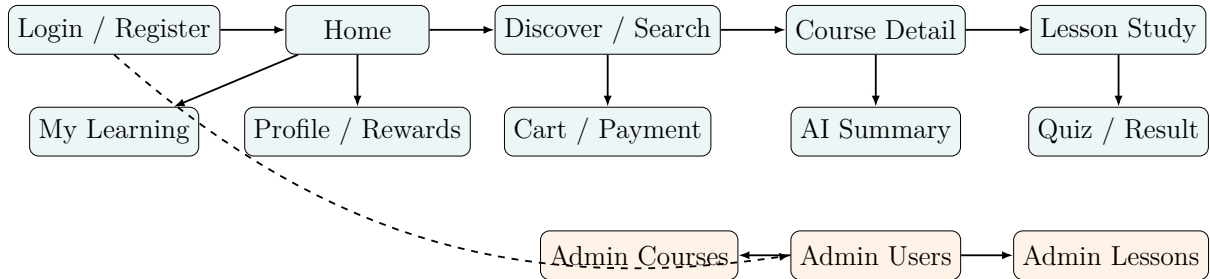


Figure 5.5: Android navigation and principal user journeys

5.5 Deployment

The base Compose stack runs the API, PostgreSQL/pgvector, Redis, and RabbitMQ. Named volumes preserve database and cache state through ordinary restarts. Health checks order startup by dependency readiness. The monitoring overlay adds Prometheus, Grafana, Alertmanager, Blackbox Exporter, PostgreSQL Exporter, and Redis Exporter.

Only selected interfaces are published to the host. PostgreSQL, Redis, exporters, and the Actuator management listener remain on the Compose network. This reduces

accidental exposure while retaining local access to the API, RabbitMQ management, and loopback monitoring tools. Figure 5.6 shows the topology.

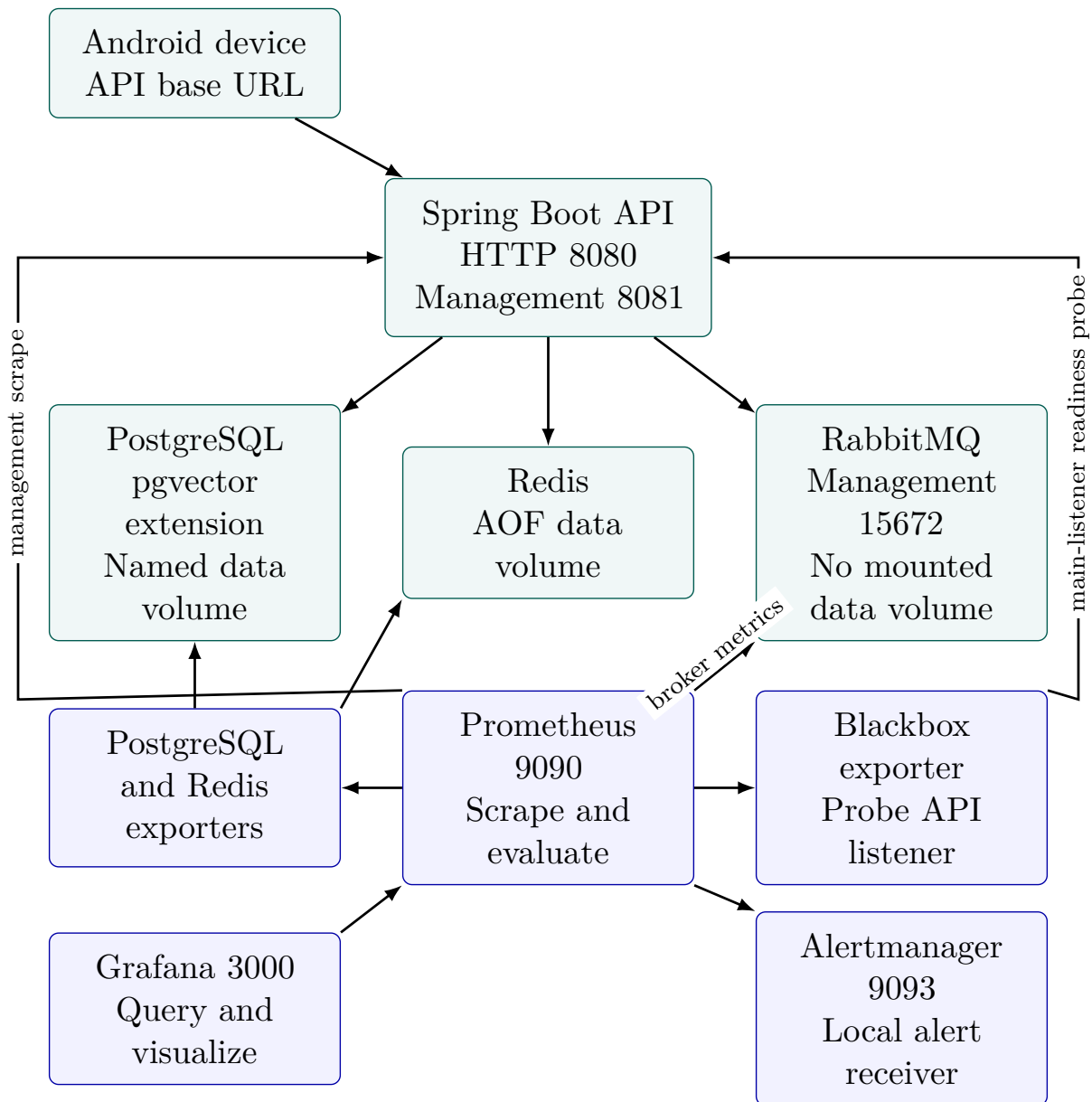


Figure 5.6: Compose service dependencies: arrows follow callers; broker data storage is not mounted

The embedding listener currently runs in the same application process as the HTTP API. The separate consumer node in the runtime figure is a logical responsibility, not an independently deployed worker container. In addition, the base Compose file persists PostgreSQL and Redis but does not mount a RabbitMQ data volume. A durable queue declaration consequently must not be interpreted as durable broker storage across container replacement.

5.6 Decisions and rejected alternatives

Table 5.1: Architectural decisions and their practical consequences

Decision	Reason and selected behavior	Cost or alternative
Keep reward writes synchronous	A successful learning command should leave a complete, inspectable reward state	A queue-based reward service would require reconciliation and a new consistency contract.
Serialize per learner in SQL	One shared lock covers related progress and XP changes across requests	Lock waits consume database connections; work inside the transaction must stay short.
Keep relational and vector data together	Course filtering and semantic ranking can share a query and lifecycle	A dedicated vector service may scale separately but adds synchronization work.
Use explicit jOOQ statements	Affected-row counts expose whether a transition actually happened	Generated schema metadata and handwritten SQL both require migration discipline.
Publish embeddings after commit	Consumers read committed content and inference does not run inside the write transaction	Publication can fail after the database has committed.
Keep Java/XML Android	Fits the project conventions and existing Material navigation	Screen state and lifecycle handling remain explicit rather than supplied by a complete reactive architecture.

These decisions favour a small number of clear failure boundaries. A later change should be justified by a measured bottleneck or product requirement, rather than by adding infrastructure solely to make the architecture appear more complex.

5.7 Request and failure boundaries

A mobile operation crosses the device-to-API boundary, the security-to-application boundary, and usually an application-to-dependency boundary. Each can fail in a different way. Connectivity failures produce a retryable mobile state. Authentication and authorization failures return 401 or 403. Invalid input returns a validation response. Transaction failures roll back related writes before an error leaves the server.

This classification prevents an empty result from being confused with an error. An empty catalogue is a successful query with no matching courses. A timeout, rejected token, or unavailable database needs a distinct user message and operational signal.

Chapter 6

Database Design

6.1 Core model

Table 6.1: Principal entity groups

Group	Stored information
Users	Identity, BCrypt hash, profile, avatar, and role.
Courses and lessons	Metadata, price, images, lesson hierarchy, content, and media URL.
Quiz content	Questions, answer options, correct answers, and attempts.
Learning state	Enrolments, lesson completion, percentage, and timestamps.
Commerce and reviews	Payment records, ratings, text, and user-course relationships.
Gamification	XP, level, streaks, badges, badge awards, quests, and quest progress.
Semantic search	Course and lesson embeddings stored as pgvector columns.

Figure 6.1 focuses on business relationships rather than listing every column. Users and courses meet through enrolments, transactions, and reviews. Courses contain lessons; lessons connect to progress, questions, and attempts. Gamification is linked to the learner while badge definitions remain reusable across learners.

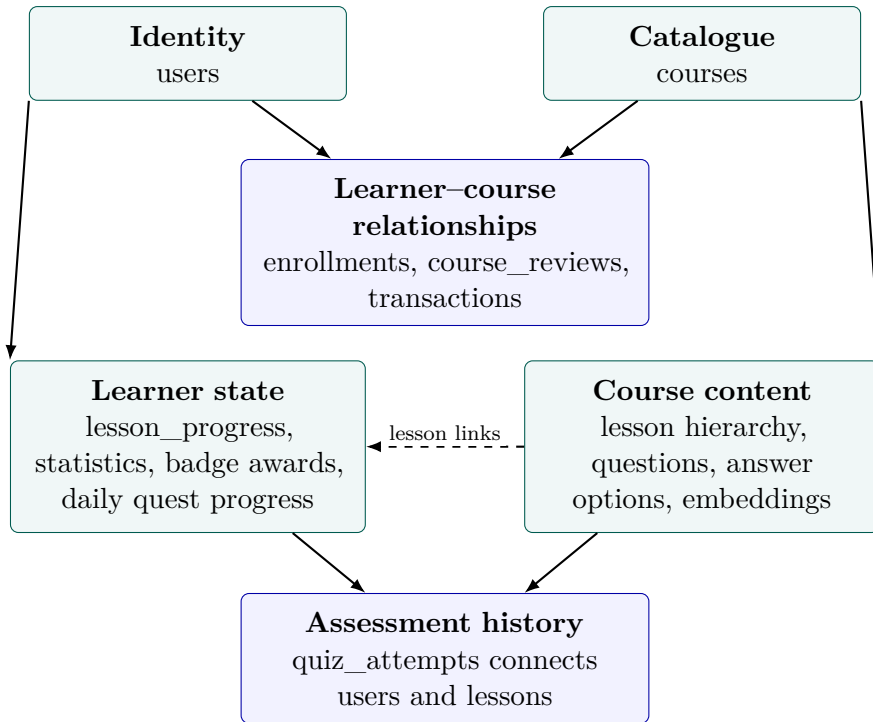


Figure 6.1: Domain relationship overview; detailed figures specify keys and cardinality

6.2 Aggregate boundaries and relational cardinality

The schema distinguishes definitions from learner activity. A course and its lessons describe reusable content. An enrolment connects one learner to one course. Progress connects one learner to one lesson. Quiz attempts retain a history of submissions, whereas a badge award represents a singular earned relationship. Combining these concepts into one table would blur the difference between history and current state.

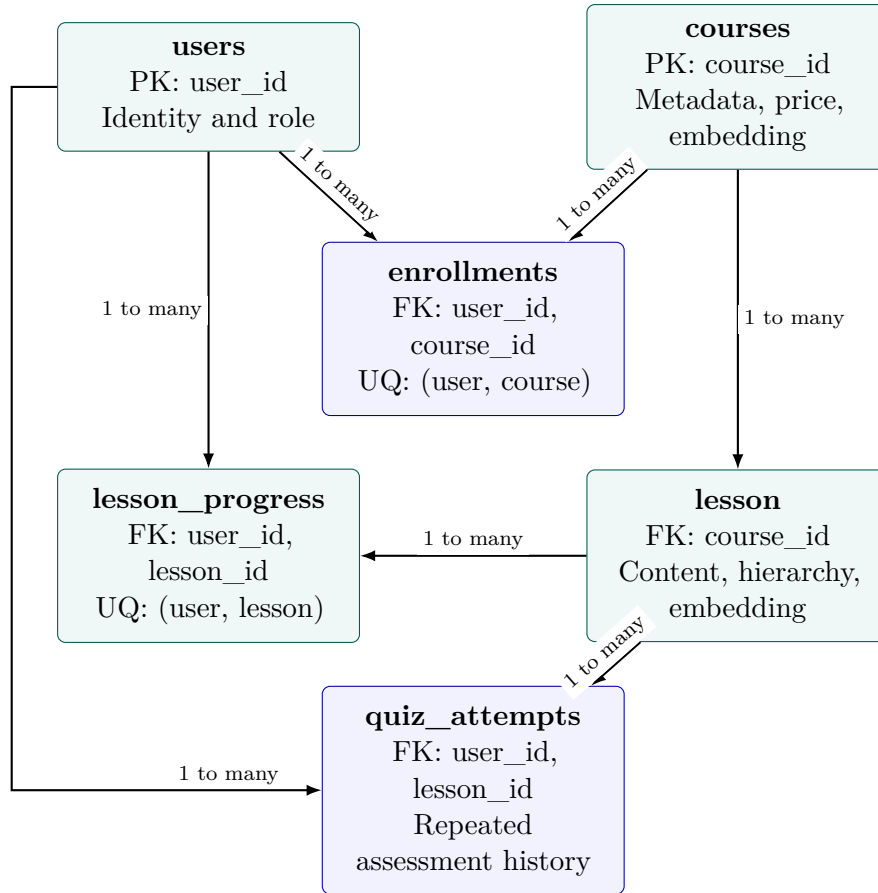


Figure 6.2: Learning data model with cardinality and natural-key constraints

The detailed model in Figure 6.2 makes the natural keys explicit. Multiple learners can use the same lesson and a learner can study many lessons, so progress resolves a many-to-many relationship. The unique user–lesson pair prevents two completion records even when callers race. Attempts intentionally have a different cardinality: a learner may submit an assessment several times. This is why single XP does not imply a single attempt row.

Identity also comes from two different places. UUID primary keys identify resources, while the verified JWT subject identifies the caller. Supplying a valid resource UUID does not prove that the caller owns the associated learner record. Referential integrity and authorization therefore solve complementary problems.

6.3 Ownership and lifecycle

The user row is the root for authentication and learner-specific records. Removing a user cascades through progress and quiz activity where migrations explicitly define that behavior. A course owns its lesson hierarchy, and a lesson may reference a parent lesson for grouped content. Questions belong to lessons and options belong to questions. This ordering lets content deletion remove dependent assessment data in a predictable

direction.

Enrolment is the bridge between catalogue data and learner activity. The backend checks it before accepting learning mutations. Progress records answer whether a specific lesson transition happened; the enrolment holds the broader course state. Gamification statistics aggregate learner rewards, while attempts retain assessment history. Keeping these concepts separate allows an attempt to be audited without recalculating an already awarded first-pass reward.

6.3.1 Assessment definitions and activity records

A lesson may contain several questions, and each question may contain several answer options. Correctness is stored on the option; the learner-facing `GetQuizResponse` returns option identifiers and text without the correctness flag. The question's `points` field is returned to the client, but the current multiple-choice grader counts correct submitted items equally. Consequently, stored points are content metadata rather than an implemented weighted scoring rule.

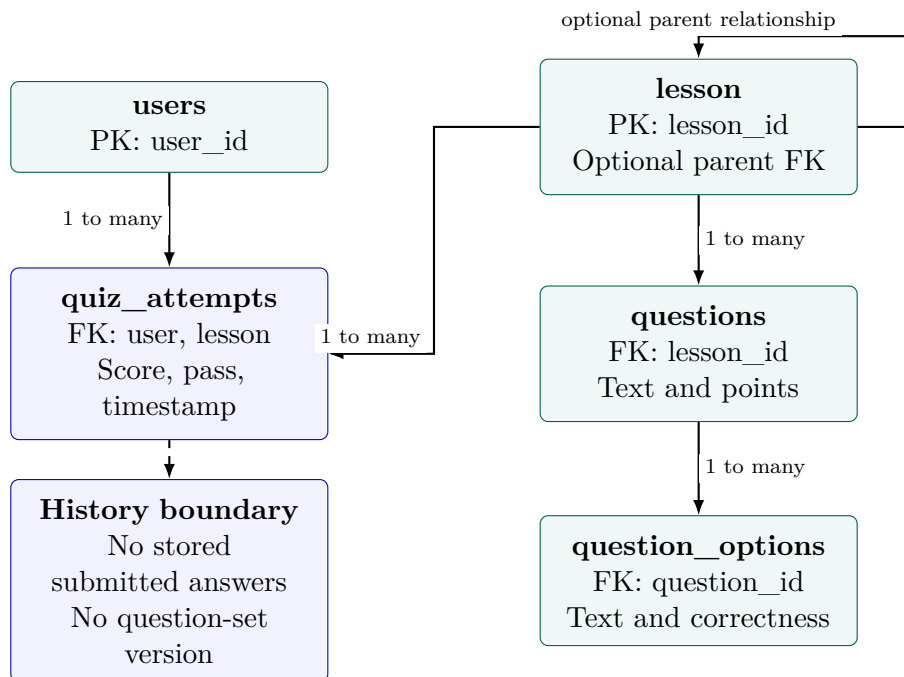


Figure 6.3: Assessment definitions and persisted history; dashed note identifies missing audit detail

Figure 6.3 separates the answer key from attempt history. The attempt table stores the learner, lesson, percentage, pass flag, and timestamp. It does not persist each submitted answer or a version of the question set. A later edit can therefore change the assessment without preserving the exact historical answer key that produced a previous score. A durable assessment audit would need versioned definitions and attempt-answer rows, rather than relying on the current question options to reconstruct an old submission.

The hierarchy is also consequential: a quiz child points to its content parent through `parent_lesson_id`. The foreign key confirms that the parent exists, but does not itself require the parent to be in the same course or prevent a cycle. Those rules would need additional validation. The diagram represents existing relationships; it does not imply those missing constraints have been implemented.

6.3.2 Reward definitions and daily state

Gamification uses a statistics row for current totals and separate definitions for badges and quests. The seeded quests are `QUIZ_COUNT` (ten submissions, 100 XP), `STUDY_TIME` (1,200 reported seconds, 80 XP), and `PERFECT_RUN` (three successive perfect submissions, 60 XP). These values come from migration 1.2.3 and can change if the stored definitions are edited. The daily progress key includes the configured local date, so the same quest can be earned on another day without resetting yesterday's record.

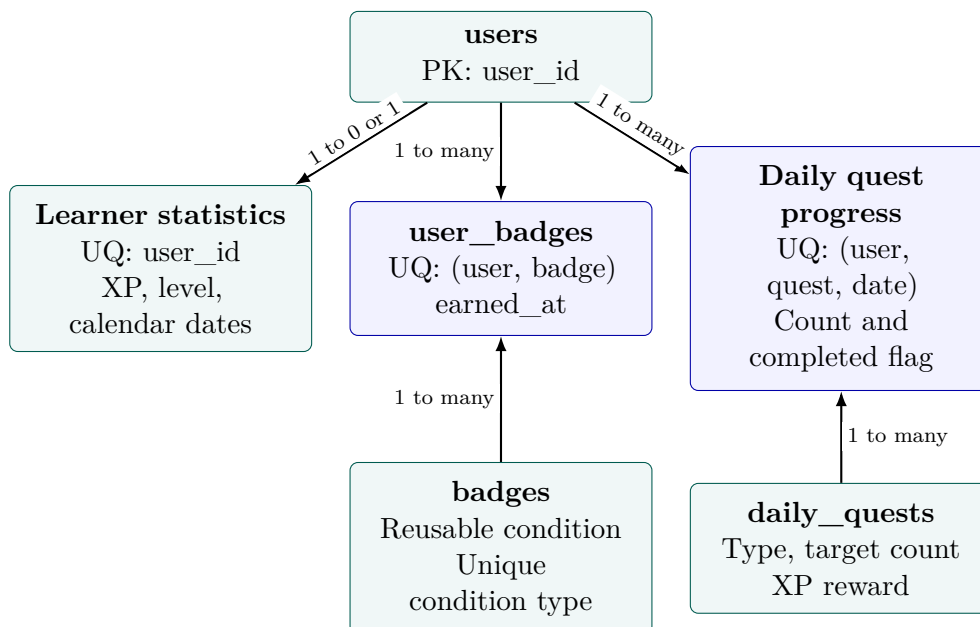


Figure 6.4: Reward definitions, aggregate totals, and per-learner earned relationships

The unique learner–badge pair protects an award, while the unique learner–quest–date tuple protects a daily counter. Neither uniqueness rule proves that the triggering learning activity actually happened. That distinction becomes important at the quest reporting endpoint discussed in Section 8.4.

6.3.3 Deletion, retention, and reserved fields

Deleting a course cascades to lessons and their progress and attempts through the declared foreign keys. Reviews and transactions have separate foreign keys whose original definitions do not specify the same cascade. Thus a generic course or user delete can be

blocked by dependent rows; the report must not promise universal cleanup. The delete services do not implement a retention workflow or an archive of historical achievements. Dynamic badge conditions encode a course UUID as text rather than a course foreign key, so deleting a course also does not automatically prove that its badge definition has been removed.

Table 6.2: Data lifecycle boundaries in the inspected implementation

Data	Existing behavior	Unimplemented policy
Learning activity	Selected foreign keys cascade after parent deletion	Historical retention and achievement policy
Reviews and transactions	Independent foreign-key relationships	Consistent deletion or archival policy across every dependent table
Embeddings	Derived vector stored with course or lesson	Versioned model migration and reconciliation
Supabase media	Database records store external object URLs	Coordinated removal of remote objects and orphan detection
Reset tokens	Digest, expiry, and consumed timestamp	Scheduled deletion of expired records
Metrics and alerts	Bounded labels; Prometheus retention is configured	Retention and access policy for application logs and backups

The schema contains an `ai_dropout_risk_score` enrolment field. Its presence is not evidence of a trained predictor or an evaluated dropout model. Likewise, completion badges labelled “Certified” are in-app achievements, not independently accredited educational certificates.

6.4 Schema evolution

The 28 migrations begin with users and progressively add courses, lessons, quizzes, enrolments, gamification, payment, reviews, media, lesson hierarchy, embeddings, quests, indexes, and constraint alignment. Fresh verification establishes that a clean database reaches version 1.2.7. It does not establish that every previously populated deployment upgrades without data conflicts. Hibernate validates mapped entities at startup, while Flyway checks migration history; neither substitutes for an upgrade rehearsal with repre-

sentative existing data. jOOQ source generation is explicit, allowing ordinary builds to run without a developer database.

Migration 1.2.7 removes duplicate successful transaction records before installing its partial unique index. This is an intentional data transformation, not merely the addition of a column. A real upgrade should preserve a backup, record the affected rows, and verify the retained payment and enrolment relationships. The current clean-database run does not measure that upgrade impact.

6.5 Integrity and connections

Foreign keys connect activity to users and content. Unique constraints protect singular relationships such as enrolments and badge awards. Indexes support common course, progress, ranking, and lookup paths. Constraints remain necessary even when application code checks for an existing row because simultaneous transactions can both pass a pre-insert check.

Table 6.3: Selected integrity rules and their purpose

Data	Database rule	Invariant protected
Enrolment	Unique user–course pair	A learner joins a course once.
Lesson progress	Unique user–lesson pair	One authoritative completion transition exists.
Gamification statistics	Unique user identifier	Concurrent initialization cannot create two totals.
Badge award	Unique user–badge pair	A badge definition may be reused but its award is singular per learner.
Course review	Unique user–course pair	A learner maintains one review for a course.
Successful checkout	Partial unique user–course index where status is SUCCESS	Retries converge on one successful simulation record.
Daily quest progress	Unique user–quest–date tuple	Daily work cannot create duplicate counters.

Data	Database rule	Invariant protected
Badge definition	Unique condition type	Concurrent course completion reuses one definition.
Dependent learning rows	Foreign keys with selected cascades	User and content deletion has a defined direction.

HikariCP defaults to a maximum pool of 10 and two idle connections. Open Session in View is disabled, preventing lazy database work during response rendering. Pool active, maximum, and pending counts are exported as metrics.

6.6 Relational and vector data together

Embedding columns live beside course and lesson records, so semantic retrieval can apply normal relational filters before similarity ordering. A course question first restricts candidate material to the requested course and only then ranks vector distance. The benefit is one transaction and backup boundary for content and vectors. The trade-off is that embedding dimensions and extension availability become part of the PostgreSQL schema and migration process.

The current MiniLM model produces 384 values. A future model change with a different dimension would require a planned migration and complete re-embedding. Recording the model identity with future vectors would make mixed-model transitions safer.

6.7 Queries, indexes, and connection budget

The application uses a HikariCP pool with a default maximum of ten connections and two minimum idle connections. Each active database transaction occupies a connection, including time spent waiting for a learner row lock. Increasing the pool without understanding query duration can move contention from the application to PostgreSQL. The 32-connection integration-test setting is intentionally different and is not a recommended runtime pool size.

Relational uniqueness supports enrolment and progress lookups. The semantic migration adds IVFFlat cosine indexes with 50 lists to course and lesson embeddings. Such an index is a search mechanism, not evidence that every query uses it or meets a latency target. Query plans should be inspected on a representative dataset, and approximate retrieval should be compared with an exact baseline before tuning lists or probes [11].

Migration `V1_2_7__secure_password_reset_and_payment_idempotency.sql` adds a partial unique index for successful user–course transactions. Failed or non-success history

does not occupy that uniqueness boundary. Before applying the index, the migration removes duplicate successful records; this is a data-changing upgrade and would require backup and audit review in a deployment with real financial records. The same migration adds hashed reset tokens with expiry and consumption timestamps.

Listing 6.1: Implemented uniqueness for a successful development payment

```
1 CREATE UNIQUE INDEX uq_successful_transaction_user_course
2   ON transactions (user_id, course_id)
3   WHERE status = 'SUCCESS';
```

Chapter 7

Mobile Application Implementation

7.1 User experience

The UI shares a Material 3 theme, colors, typography, cards, and bottom navigation. Home presents learner status and continuation. Discover and Search expose the catalogue. My Learning shows enrolments. Course Detail links to study, reviews, and AI assistance. Profile and gamification screens present personal progress.

The screen review looks for loading, empty results, and retryable failures, with source evidence for representative catalogue and learning screens. These states help prevent a blank screen from being interpreted as success or a crash; their complete device coverage remains an evaluation task.

The learner journey in Figure 7.1 connects UI decisions to backend state changes. Free and paid courses converge on one enrolment concept. Content completion and multiple-choice first passes converge on protected progress updates. Fill-in-the-blank currently provides feedback through a separate, less complete backend path. The client should render the distinction instead of assuming that every successful quiz response proves a course-state transition.

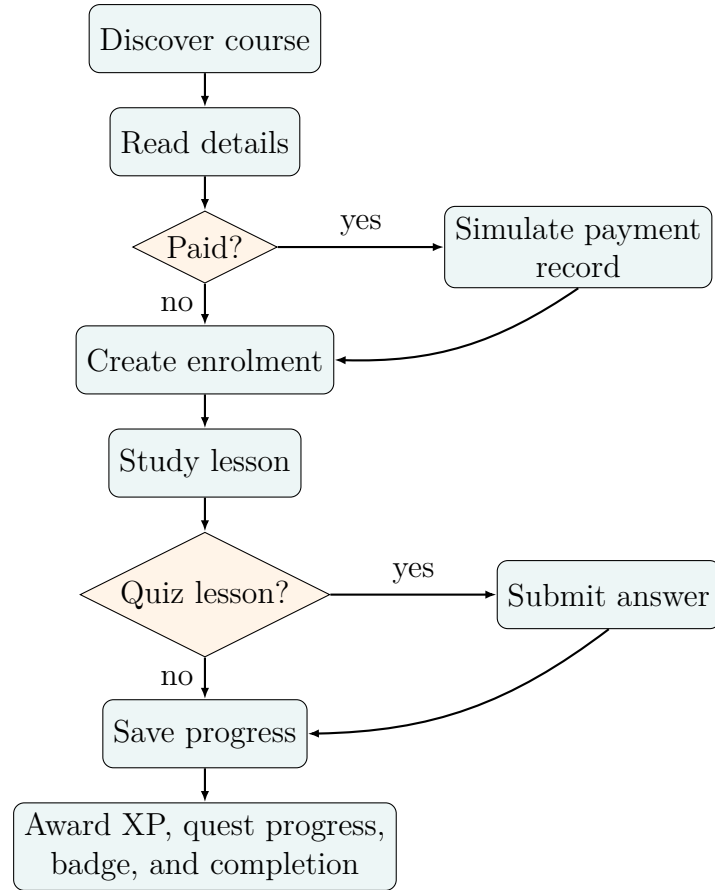


Figure 7.1: End-to-end learner course journey

After reward evaluation, an incomplete course returns the learner to the next available lesson; completed courses retain their terminal completion state.

Table 7.1: Implemented learner screens and required visible states

Screen	Successful content	Non-success state
Home	Statistics and continuation cards	Loading, absent progress, retryable request failure
Discover/Search	Course cards and filtered results	Loading, empty result, clear query, retry
My Learning	Enrolled course list	Empty route back to discovery, retry
Course Detail	Metadata, enrol/study, reviews, AI entry	Loading, missing course, network failure
Lesson/Quiz	Content, question, answer feedback	Validation, submission failure, preserved context
Profile/Rewards	Profile, XP, quests, badges	Loading, empty rewards, retry

The table is a source-level inventory, not a usability result. The recorded emulator journey observed authentication, Home, Learn, and successful authenticated calls, but it did not capture submission-quality screenshots or cover every state. A final report submission should include annotated device captures only after repeating the journey with non-sensitive demonstration data.

7.2 Networking and authentication

The API URL is injected into `BuildConfig`; the emulator default is <http://10.0.2.2:8080/>. Retrofit interfaces map each backend domain. Debug builds enable HTTP logging and release builds disable it. After login, session management stores the access token, refresh token, role, and profile information. Provider secrets are never placed in Android build configuration.

The token manager gives networking code one location for bearer credentials. API interfaces are divided by domain so course browsing does not depend on administrator or payment details. Request and response models mirror the wire contract. This design is intentionally pragmatic for the project scale: fragments coordinate their screen state and adapters render collections, while authoritative rules remain on the server.

When a request fails, the screen retains enough context to retry without losing the selected course. Authentication failures lead back to a valid session path; ordinary network failures keep a retry action close to the failed content. Debug HTTP logging supports development diagnosis, and disabling it in release avoids routine token and payload exposure in release logs.

7.2.1 Coordinating concurrent token refresh

A mobile screen can issue several requests while its access token expires. If every 401 independently refreshes the session, the device creates unnecessary network calls and races to update local credentials. `TokenRefreshCoordinator` synchronizes the refresh decision. After entering the synchronized method, a caller compares the token it sent with the token currently stored. If another request already installed a replacement, the caller reuses that value; otherwise it performs the refresh.

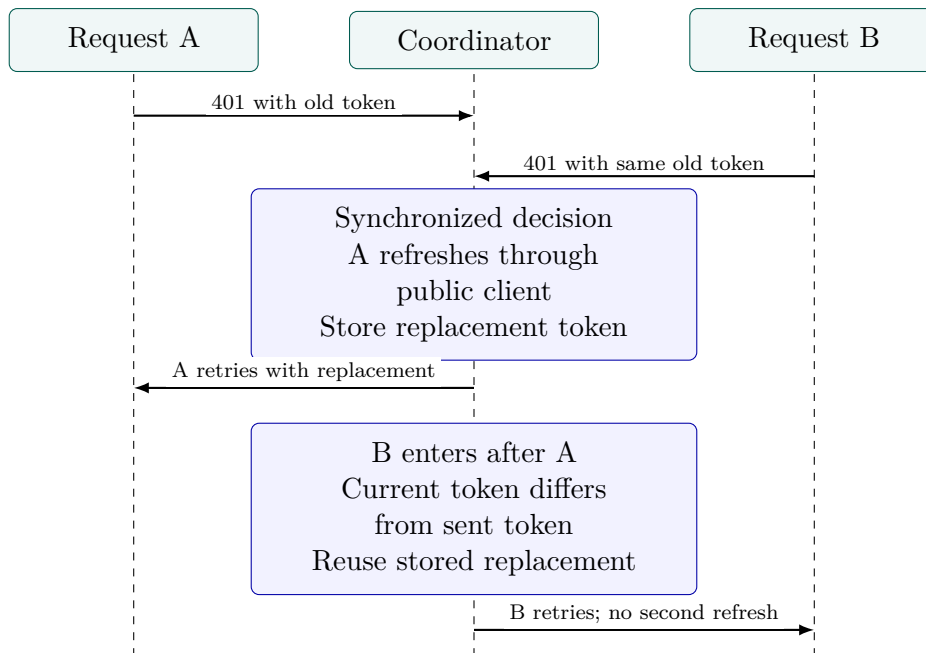


Figure 7.2: Implemented in-process coordination for concurrent expired-token requests

The OkHttp authenticator bounds the retry chain to avoid an infinite refresh loop. The refresh request uses the unauthenticated Retrofit client so that it does not invoke its own authenticator recursively. An I/O failure is distinguished from an invalid refresh response: transient connectivity does not immediately erase the saved session. The existing Android unit test exercises concurrent expired requests and verifies that they share one refresh call. This is an in-process coordination property; it does not claim synchronization across devices or processes.

7.2.2 Transport budgets and device state

The clients configure a ten-second connection timeout, thirty-second read and write timeouts, and a forty-five-second total call budget. These values bound waiting on a dependency; they do not define a measured user-facing performance objective. The ten-megabyte OkHttp disk cache follows HTTP cache semantics and should not be confused with a complete offline course store.

Local session data supplies navigation and bearer credentials, but the backend remains the authority for enrolment and rewards. The optional `X-User-Id` request header is not proof of identity: ownership checks use the authenticated principal. A production mobile threat review should also examine credential storage, device compromise, and TLS configuration rather than assuming that JWT usage secures every client boundary.

The inspected main manifest enables cleartext traffic and application backup; the token manager persists session values in ordinary `SharedPreferences`. Those are concrete development defaults, not an encrypted transport or credential-storage guarantee. A production variant needs a deliberate HTTPS/network policy and a review of backup

and local session handling. Release HTTP logging being disabled addresses one disclosure path while leaving these separate policies unresolved.

Study-time reporting also has a device lifecycle boundary. The lesson fragment records elapsed foreground time between `onResume` and `onPause`, then posts the measured seconds to the quest endpoint. The callback logs a network failure but does not persist a retryable event. This measures foreground elapsed time rather than attention, and it can lose a contribution when delivery fails. A future offline outbox for study events would need server deduplication as well as persistent client storage; replay without an event identifier could otherwise credit the same interval twice.

7.3 Learning and engagement

Lessons render content or open multiple-choice and fill-in-the-blank quiz flows. Multiple-choice answers are graded on the backend, where first-pass rewards and completion are coordinated. Fill-in-the-blank returns answer feedback but does not currently persist an attempt or award those rewards. Gamification displays XP, levels, streaks, quests, badges, and leaderboard position. WorkManager provides local reminders; optional Firebase Messaging provides remote notifications.

Course progress counts top-level lessons. A first multiple-choice pass can mark both the quiz child and its parent complete; a child is excluded from the percentage's denominator. This is the implemented learning policy, not proof that every assessment in a course has been completed. The client submits the event and renders returned state. The reliability chapter explains this calculation and its trade-offs.

7.4 UI quality and accessibility considerations

Material components provide consistent touch targets, typography, colors, and control behavior. Lists use dedicated empty views rather than leaving an empty RecyclerView. Search supports keyboard submission, clearing, and retry. My Learning offers a route back to discovery when the learner has no enrolments.

The acceptance checklist includes normal and enlarged fonts, keyboard visibility, back navigation, loading, empty, and offline states. Further accessibility work should add automated contrast and content-description checks plus TalkBack traversal tests. Those checks are proposed work and are not claimed as completed evaluation.

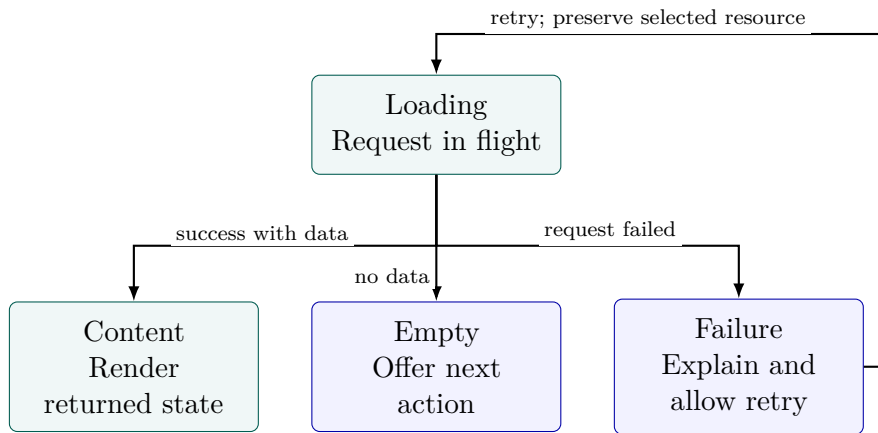


Figure 7.3: Screen-state acceptance model for loaded data; device coverage remains partial

Figure 7.3 documents the screen-state contract used to review data screens. Loading means that the outcome is still unknown; empty means that a successful query found no records; failure means that the query did not establish a result. After a learning submission, the client should render authoritative returned state rather than increment XP locally. During a retry it should preserve the selected course and lesson so the learner can recover without repeating navigation. The diagram is an acceptance model, not a claim that every branch has been exercised on every Android screen.

7.5 Build variants

The debug APK is installable for development. The release variant enables minification, optimization, and resource shrinking. CI verifies release APK and AAB generation, but the artifacts remain unsigned and are not store-ready.

Chapter 8

Backend and Security Implementation

8.1 REST API

Routes cover users, authentication, courses, lessons, enrolment, progress, quizzes, payments, reviews, media, gamification, badges, quests, and administration. Swagger UI is available at `/swagger-ui/index.html` and OpenAPI at `/v3/api-docs`. Bean validation and centralized advice provide common input and failure handling for selected contracts; validation coverage and response shapes are not yet uniform across all controllers.

The API is organized around resources rather than individual screens. Android may change its navigation without renaming the progress or course contract. Read routes return DTOs shaped for clients; commands accept JSON, form data, or multipart content according to the operation. Figure 8.1 groups routes by the security decision that protects them.

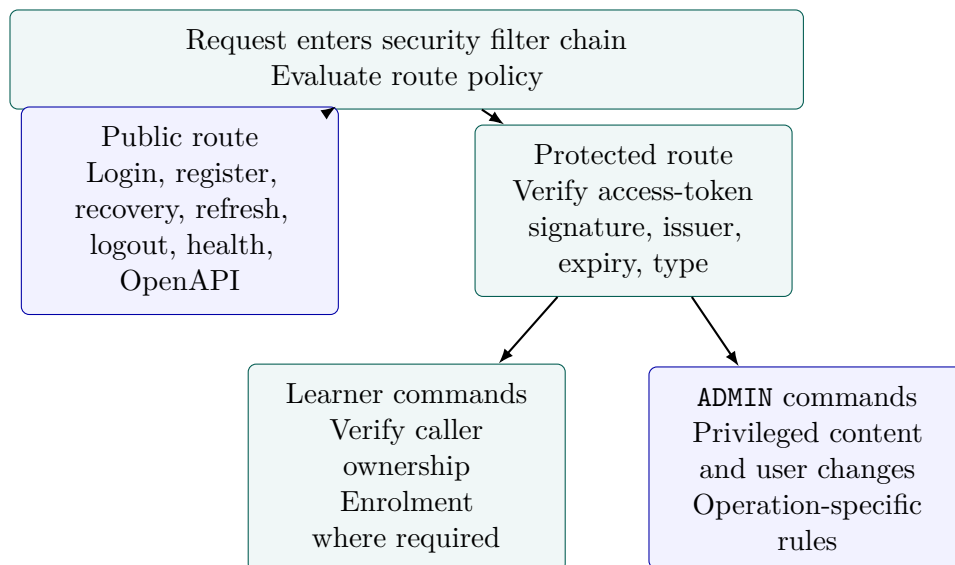


Figure 8.1: API groups and authorization boundaries

8.2 API contract groups

Account contracts create sessions and maintain profiles. Catalogue contracts return courses, search results, owned courses, reviews, summaries, and grounded answers. Learning contracts cover enrolment, lesson hierarchy, progress, and the two quiz submission forms. Gamification contracts return statistics, check-in results, quests, badges, and leaderboard entries. Media uses multipart requests because binary content has different validation and size requirements from JSON.

Administrative commands are collected under `/api/admin` for list, create, update, and delete workflows, with additional privileged mutations explicitly covered by security rules. The complete generated contract remains available through OpenAPI; the report groups related endpoints so the reader can understand responsibility and access without reproducing a large machine-generated document.

Table 8.1: Representative API contracts

Operation	Contract, success, and failure boundary
Login	POST <code>/api/user/login</code> accepts email and password. Success returns access and refresh tokens, user, and role; validation or invalid credentials fail.
Refresh	POST <code>/api/auth/refresh</code> accepts a refresh token. Success returns a new access token; an unknown user or malformed token returns 400, and an expired or revoked token returns 401.
Enrol	POST <code>/api/enrollment/{courseId}/register</code> accepts the user ID. Success creates the singular learner-course enrolment; unauthenticated and wrong-owner calls are rejected.
Complete lesson	POST <code>/api/progress/lesson</code> accepts form fields <code>lessonId</code> and <code>userId</code> . Success returns the progress percentage after a protected transition; invalid ownership, enrolment, or lesson state fails.
Submit quiz	POST <code>/api/quiz/submit-multiple-choice</code> accepts user, lesson, and answers. Success returns score, course progress, XP, and completed quests; a response-loss retry may add an attempt.
Ask course	POST <code>/api/course/{courseId}/ask</code> accepts a question of at most 500 characters. Success returns an answer, lesson sources, and provider flag; provider failure uses the fallback.

The progress command remains form-encoded for compatibility with the Android client. A representative successful response is:

Listing 8.1: Lesson-completion response contract

```
1 {  
2   "success": true,  
3   "message": "...",  
4   "newProgressPercent": 50.0  
5 }
```

The omitted message is human-readable and varies with the transition. Clients should use the boolean and percentage fields rather than parsing reward numbers or course completion out of message text. The example shows field shape, not a captured production response. The request has no client idempotency key. Protected idempotency comes from the unique user-lesson row and conditional transaction. Quiz attempts have a different boundary and can still duplicate history after a lost response, as discussed later.

8.3 Assessment execution and grading boundaries

The two submission endpoints share caller ownership checks but do not share the same application workflow. Multiple choice invokes a transactional use case that checks enrolment and lesson type, grades option membership, records an attempt, updates daily quests, and conditionally awards first-pass benefits. Fill-in-the-blank compares submitted text with stored correct option text after trimming and ignoring case. It returns per-question feedback without executing the transactional progress workflow.

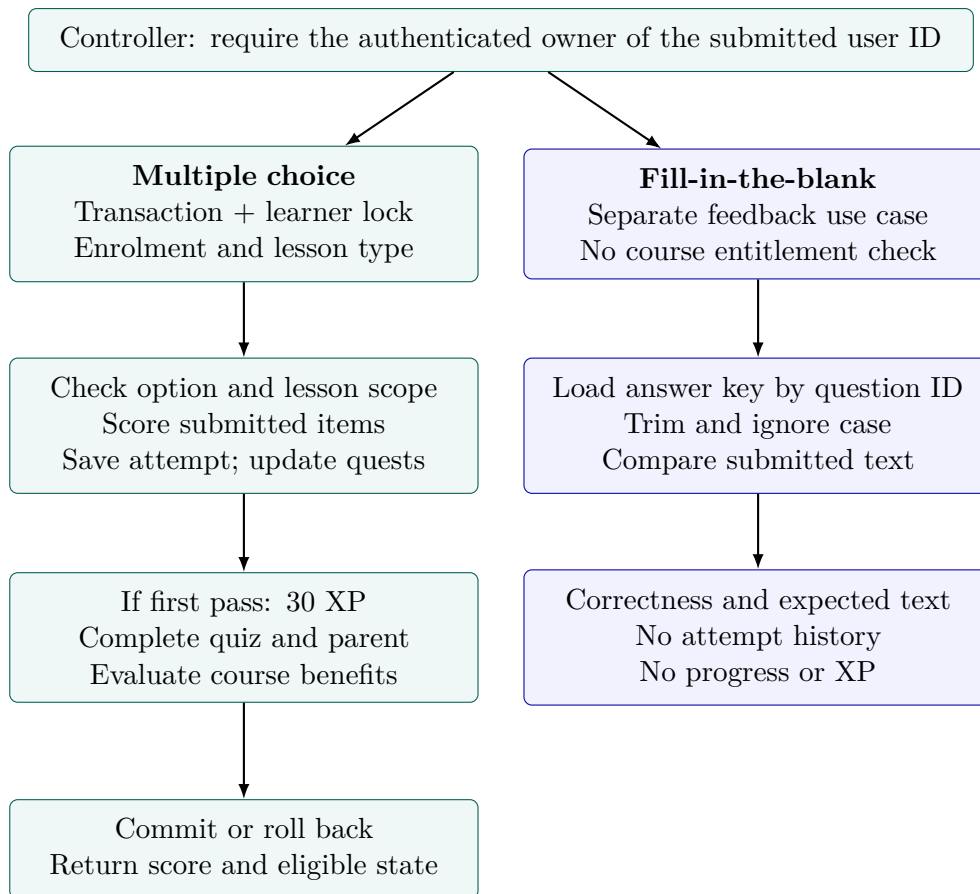


Figure 8.2: Existing quiz workflows differ in scope, persistence, and rewards; grading gaps remain

Table 8.2: Implemented assessment behavior by submission type

Concern	Multiple choice	Fill-in-the-blank
Caller identity	JWT owner check in controller	JWT owner check in controller
Resource scope	Enrolment, quiz lesson, question–option membership	Submitted question IDs; no enrolment or lesson–membership check in the use case
Grading	Equal item count; pass at 50%; percentage rounded to one decimal	Trimmed, case-insensitive exact text comparison
History	Every valid submission records an attempt	No attempt persisted

Concern	Multiple choice	Fill-in-the-blank
Rewards	First pass gives 30 XP; quests evaluated on every submission	No first-pass XP or quest update
Progress	First pass completes quiz and any parent lesson	No lesson or course update
Feedback	Score, pass flag, optional course state and completed quests	Per-item correctness and expected answer

The multiple-choice SQL joins the supplied option to its question and the selected lesson, preventing a correct option from another lesson from being counted as a correct response. This is useful scope protection, but the use case sets `totalQuestions` to the size of the submitted list. It does not compare that list with the authoritative complete question set or reject repeated question IDs. For a four-question quiz, a submission containing one correct item is consequently computed as $1/1 = 100\%$, rather than $1/4 = 25\%$. Repeating a correct question can similarly distort the assessment and perfect-run count. These examples are deduced from the inspected grading loop, not presented as new live HTTP test results.

Correct grading should first resolve the server-owned question set, require exactly one allowed answer per required question, reject unknown and duplicated identifiers, and then calculate the score against that set. Whether questions have equal weights or use stored points should be an explicit contract. No concurrency mechanism can repair a score computed from an invalid submission.

The fill-in-the-blank path has additional gaps. Its `lessonId` is not used to restrict question lookup, and missing answer-key data can reach a null list before feedback is built. That is a server-error risk rather than a deliberate “question not found” response. The request also lacks the equivalent bean-validation coverage. These are reported implementation defects, not limitations inherent to text questions. The next implementation should reuse the validated assessment scope and define whether this mode is practice-only or earns the same persisted progression.

8.4 Activity trust and overlapping contracts

An owner check establishes whose record a caller can change. It does not establish that the caller studied, attempted a quiz, or measured time honestly. The daily quest progress endpoint currently accepts a `questType` and arbitrary integer `increment` from the authenticated owner. The service can award a quest when the resulting count reaches its target.

This preserves one completion reward per day, but allows client-reported activity to qualify without server-observed evidence. The explicit XP-grant route is administrator-only; the quest reporting route is a separate trust boundary and must be evaluated independently.

A stronger policy would derive quiz and perfect-run contributions only from validated, uniquely identified assessment submissions. Study-time credit needs a defined evidence model, bounded intervals, and replay handling. Even a heartbeat cannot prove learner attention, so the product should describe reported study time honestly. Merely adding a maximum increment would restrict obvious abuse without fully establishing activity authenticity.

There are also overlapping review contracts. POST [/api/course/reviews](#) checks the request owner and enrolment. POST [/api/course/{courseId}/reviews](#) obtains the user from the JWT and validates the course and review fields, but its use case does not check enrolment. They share the review storage while applying different eligibility rules. A canonical policy and route should be selected before claiming that all course reviews are restricted to enrolled learners.

Authenticated catalogue access should not be confused with course entitlement. GET [/api/lesson?courseID=...](#) and GET [/api/quiz/{lessonId}](#) have no course-enrolment check in their inspected controller/use-case paths. AI assistance does check enrolment, and progress mutations do check it. If paid courses are intended to protect content delivery as well as learning mutations, these reads require the same policy. The current development checkout is not a complete commercial access control system.

8.5 Authentication and authorization

Passwords use BCrypt. Login returns a 15-minute access token with identity, email, role, and token type, plus a 30-day refresh token with a unique identifier. Refresh state is held in Redis and logout revokes it. Spring Security uses stateless sessions. Public routes include registration, login, recovery, refresh, logout, health, and API documentation. Other routes require authentication. Administrator operations require the **ADMIN** role; learning mutations also validate ownership.

Figure 8.3 follows both the initial login and later refresh. The access token is self-contained and short-lived. The refresh token is longer-lived, but its identifier is checked in Redis, which gives logout a server-side revocation point. A refresh token is not accepted as an access token because the signed token also carries a token-type claim.

Logout deletes the selected refresh-token record. It prevents that refresh token from minting another access token; it does not immediately invalidate an already issued 15-minute access token. This bounded residual session is a consequence of stateless access-token validation.

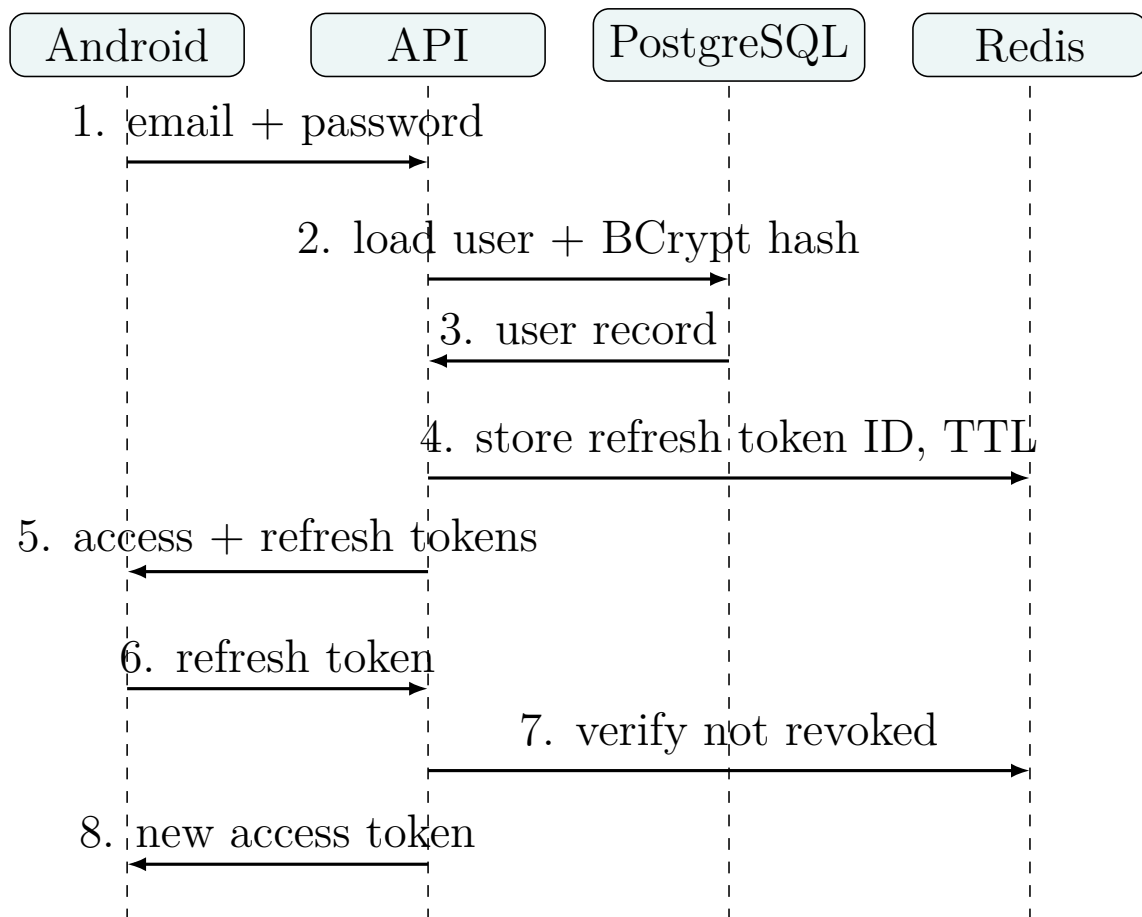


Figure 8.3: Login and access-token refresh sequence

Authorization has two levels. Route rules decide whether a role may reach an endpoint. Application rules then check whether the authenticated learner owns the requested user path and is enrolled in the relevant course. Role checks alone cannot protect a request such as “save progress for user B” made by authenticated user A.

8.6 Single-use password recovery

Account recovery is a separate credential lifecycle rather than a direct password update keyed by an email address. When delivery is configured, the service generates 32 random bytes with `SecureRandom` and encodes them as a URL-safe token. Only the SHA-256 digest is stored in PostgreSQL; possession of the stored digest alone does not provide the code submitted by the learner. The default lifetime is fifteen minutes.

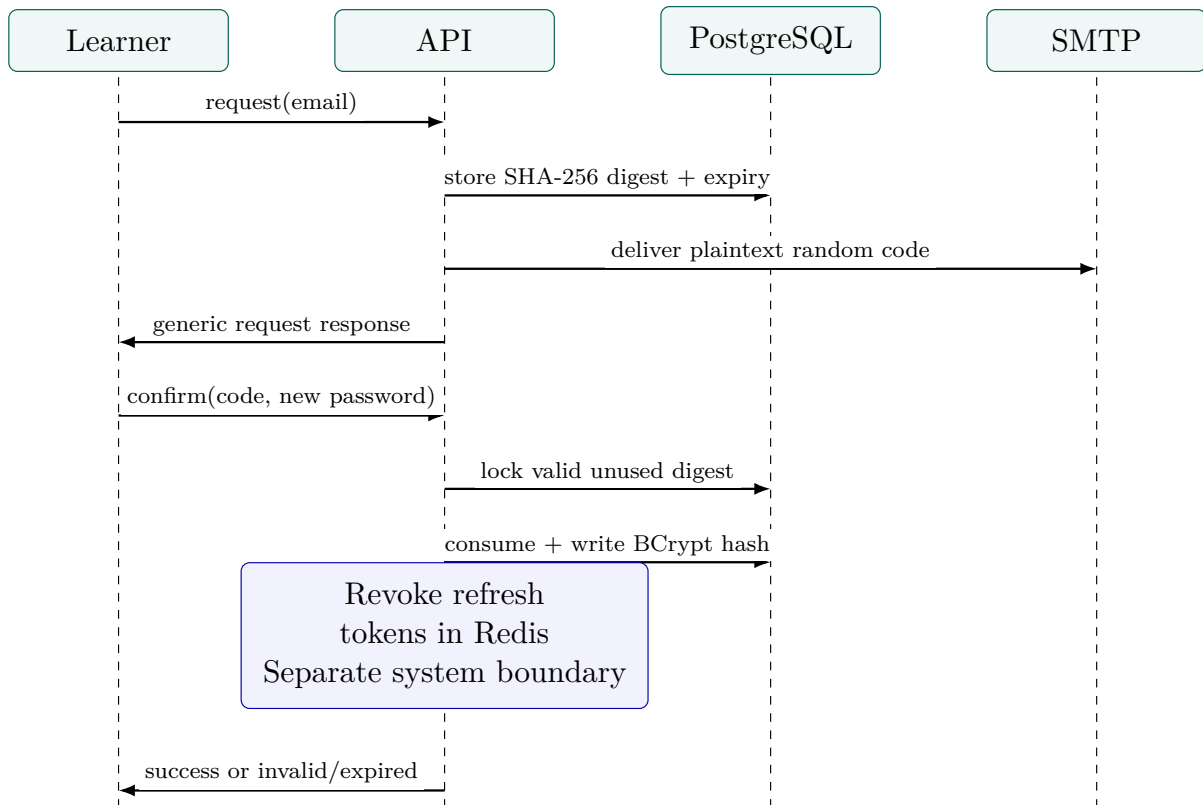


Figure 8.4: Implemented recovery lifecycle when SMTP is configured; not one distributed transaction

The request response uses the same message for an existing and an unknown account, reducing direct account enumeration through response text. When no delivery bean is configured, recovery explicitly reports unavailability. SMTP is therefore an optional integration with a visible unavailable state, rather than a pretend successful reset. On confirmation, the repository locks an unused, unexpired token row and conditionally marks it consumed. The service then writes a BCrypt hash and revokes the user’s refresh tokens. The reset-token integration test covers single use and expiry; SMTP delivery and concurrent recovery requests have not been demonstrated end to end.

The transaction spans the PostgreSQL token and password changes, but it does not form a distributed transaction with SMTP or Redis. For example, a delivered email can outlive a database rollback, and Redis revocation cannot be undone by PostgreSQL. Rate limiting, delivery retries, a session-version strategy, and explicit failure tests remain useful next steps. Issued access tokens retain their normal expiry boundary.

8.7 Development checkout and enrolment integrity

The payment screen demonstrates a checkout journey; it is not a live MoMo integration. `ProcessPaymentUseCase` is gated by a simulation flag. Direct backend execution defaults the flag to false, while the local Compose file defaults it to true. The response explicitly

marks simulation and says that no real payment method was charged.

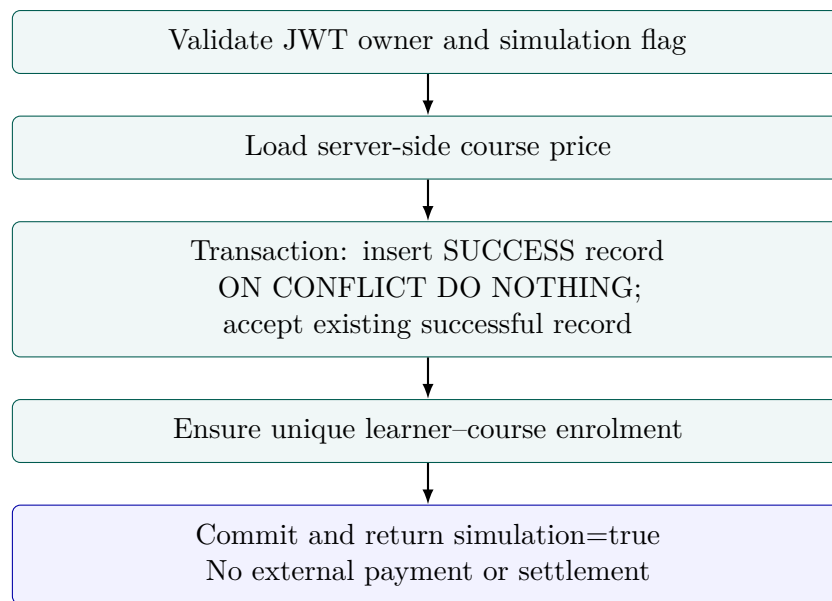


Figure 8.5: Retry-safe development checkout; repeated requests converge on one successful record

The controller checks ownership before the service reads the authoritative course price. Within one transaction, the repository inserts a successful record using `ON CONFLICT DO NOTHING`. If a successful record already exists, the service can return the existing outcome and ensure enrolment exists. The partial unique index prevents duplicate successful rows for one user-course pair; the enrolment constraint protects the corresponding access relationship. Paid courses reject the free-enrolment route. The integration suite verifies repeated simulation and wrong-owner rejection, not a real payment provider or a concurrent payment stress test.

The schema’s legacy `MOMO` method value names a demonstration option, not evidence of external settlement. A real payment implementation would require verified provider callbacks, provider transaction identifiers, amount and currency checks, a state machine for pending/succeeded/failed/refunded outcomes, and reconciliation. Those responsibilities cannot be replaced by a client-selected success flag.

8.8 Contract errors and authorization threats

Authentication and resource ownership are evaluated before sensitive commands. A request without a usable bearer token is rejected with 401; an authenticated caller outside the permitted role or ownership boundary receives 403. The global advice maps validation errors to field-specific 400 responses and missing resources to 404. Some use cases instead return an operation-specific `success=false` DTO, and filter errors are produced outside

the controller advice. Consequently, the current API has several response shapes rather than one universal error envelope.

Table 8.3: Concrete threats, implemented controls, and residual limits

Threat	Implemented boundary	Residual limit
Cross-user record mutation	Compare request user ID with verified JWT subject	Route-by-route audit remains necessary.
Learner grants own XP or edits content	ADMIN route restrictions and owner checks	Administrator provisioning and audit trail need deployment policy.
Fabricated quest activity	Owner check and one daily completion transition	Client increment is trusted; server-derived activity is needed.
Incomplete or repeated quiz answers	Question–option–lesson join	Complete question coverage and distinct IDs are not enforced.
Refresh token used as bearer access	Signed token-type validation	A stolen access token lasts until expiry.
Access to paid course AI	Require enrolment, or administrator authority, before retrieval	Content distribution and entitlement need broader review.
Reuse of a reset code	Digest storage, expiry, row lock, conditional consumption	SMTP, enumeration timing, and rate limits need separate evaluation.
Duplicate successful checkout	Partial unique transaction index and transactional enrolment	Simulation has no real settlement or refund state.
Sensitive values in telemetry	Bounded outcome labels and release HTTP logging disabled	Operational logs and external provider handling still need review.

8.9 Media

The server can upload avatars, course images, and lesson videos to Supabase Storage when configured. The secret stays on the backend. The core service can run without these optional credentials.

8.10 Redis cache policy

Table 8.4: Configured Redis cache lifetimes

Courses	5 min	Lessons	5 min
Quizzes	10 min	Leaderboard	1 min
User statistics	2 min	Badge catalogue	30 min
User badges	5 min	Reviews	3 min
Course catalogue	5 min	AI summaries	6 hours
Semantic search	15 min		

Selected writes evict named entries. Embedding processing clears semantic and summary results after changed content. Cache statistics support measurement. The table lists configured lifetimes; it does not mean that every named cache has an active reader. Inspected cacheable paths use courses, course catalogue, lessons, semantic search, and AI summaries. Configured quiz, statistics, leaderboard, badge, and review cache names alone do not demonstrate caching of those endpoints.

Figure 8.6 shows the cache-aside behavior. A cache hit avoids a database query. A miss loads authoritative data and stores a bounded copy. Mutations evict domain caches rather than attempting to modify several serialized cached representations in place.

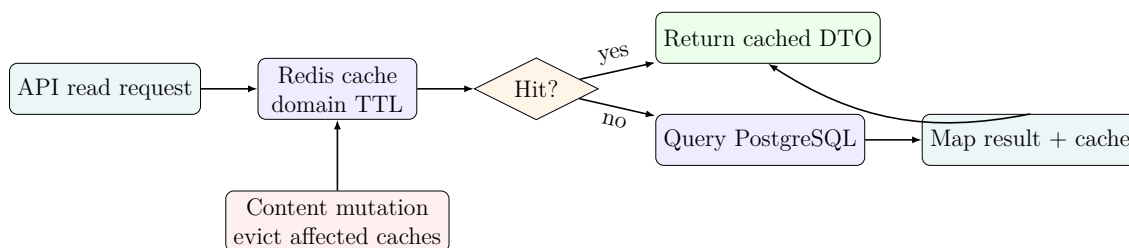


Figure 8.6: Cache-aside read and invalidation flow

The configured lifetimes are shorter for changing learner state and longer for badge definitions and generated summaries. This is an intended consistency policy, with only selected readers currently connected to it. Explicit eviction can narrow the stale interval after known writes; expiration bounds the lifetime of each existing entry. A TTL does not by itself impose a maximum cache size under unbounded new keys.

8.10.1 Cache invalidation and consistency boundary

The configured cache manager is transaction-aware, so transaction-bound cache changes can be deferred until commit. This avoids evicting or publishing derived cache state as though an authoritative write had succeeded when its database transaction later rolls back. The embedding listener currently clears the entire semantic-search and summary caches, rather than selecting only one changed course.

The inspected eviction lists are incomplete. Course listing is cached twice: an outer `courses` response keyed by user and an inner `courseCatalog` query keyed by administrator/active view. Course writes evict the inner catalogue, summary, and search entries, but do not evict the outer `courses` response. The client can consequently continue receiving an old outer result until its five-minute TTL expires. Lesson creation clears summary and search caches but omits the cached lesson list; lesson update and deletion do clear that list. This is a specific missing-invalidation issue in addition to the generic concurrent-fill race. Search also excludes already-enrolled courses, but successful enrolment does not invalidate that learner’s cached search results. A cached suggestion can remain visible for up to its fifteen-minute lifetime after the learner joins the course.

A regression should warm the actual route, mutate the content, immediately reread it, and verify the new version without waiting for expiry. Every outer and inner representation must appear in a mutation-to-cache dependency map. Clearing a different cache name is not sufficient simply because both caches describe courses.

Annotation-based caching also has an important implementation constraint: calls must cross the Spring proxy for the annotations to apply [23]. An annotation on a self-invoked method is not evidence of a cache hit. Similarly, enabled cache statistics describe instrumentation capability; a recorded hit ratio is needed before claiming fewer database reads. Fallback summaries are explicitly excluded from caching by the service’s `unless` condition, and the course-question method itself is not annotated as a cached answer operation.

TTL expiration bounds the lifetime of a stored copy, not every race. A reader that loads old data just before a commit may repopulate a stale copy after eviction. Versioned keys, coordinated cache loading, and a staleness test are possible future controls. No cache-stampede prevention or general Redis fail-open policy is claimed.

Chapter 9

Reliability, Concurrency, and Messaging

9.1 Transactional updates

Content completion coordinates progress, XP, streak, course state, and badge evaluation. Multiple-choice submission additionally records assessment history and updates daily quests within its transaction. Fill-in-the-blank feedback does not currently execute those state changes. An injected failure after intermediate writes verifies rollback for the tested transactional path, so a later retry can complete its protected transitions once.

The rollback argument is scoped to transaction boundaries and exception behavior. With Spring's default declarative rules, an unchecked exception reaching the transaction interceptor triggers rollback [24]. Catching an exception inside a transactional service and returning a failure DTO is not, by itself, a rollback instruction. For example, the lesson-creation service catches a broad exception while creating a main lesson, its quiz child, and content events. That path needs a dedicated failure test or an explicit rollback policy before the same all-or-nothing claim can be extended to every content command. The existing injected rollback test establishes its own exercised learning path.

9.1.1 Invariants rather than successful responses

The design starts from final-state rules. A content lesson has at most one user-lesson progress row. A successful first completion contributes 20 XP once. A repeated completion preserves its timestamp. Course completion and badge awards must not depend on whether the final content lesson or final quiz commits first. These are database properties, not claims about how many HTTP responses return 200.

If two requests both read an incomplete state before either writes, an application-only check can award XP twice. EduFlex therefore creates the statistics row with `ON CONFLICT DO NOTHING`, locks it in the transaction, and uses affected-row counts to decide

whether each transition is new. PostgreSQL holds conflicting row locks until transaction completion, which makes the serialized decision visible to the next writer [16].

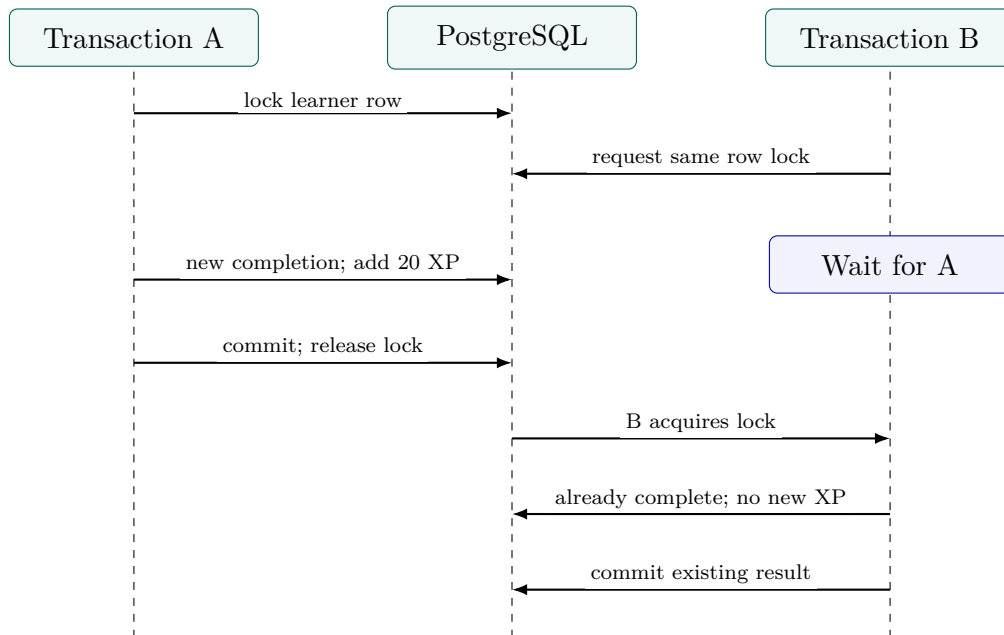


Figure 9.1: Successful duplicate completion ordering for one learner under a row lock

Figure 9.1 follows the same learner through two calls. The second waits for the first transaction, then observes completion and skips the protected reward. If the first transaction fails, the second can instead become the first committed completion. Calls for different learner rows do not acquire this same logical lock.

Listing 9.1: Implemented per-learner lock initialization and acquisition

```

1 INSERT INTO gamification_stats (user_id, xp, level, streak_days)
2 SELECT user_id, 0, 1, 0 FROM users WHERE user_id = ?
3 ON CONFLICT (user_id) DO NOTHING;
4
5 SELECT * FROM gamification_stats
6 WHERE user_id = ?
7 FOR UPDATE;

```

The transaction updates XP arithmetically in SQL rather than performing a detached read-modify-write in Java. Level is derived from the resulting XP with $\lfloor XP/100 \rfloor + 1$. Conditional course completion determines the single transaction allowed to award completion benefits. The test suite deliberately changes the content/quiz ordering to verify this rule.

9.2 Per-learner serialization

The service locks the learner’s statistics row with PostgreSQL `SELECT FOR UPDATE`. Requests for one learner serialize at the critical transition, while different learners proceed independently. Conditional updates decide whether a transition is new and uniqueness constraints provide a final safeguard. Protected transitions include lesson completion, daily check-in, first quiz pass, course completion, quest completion, and badge award. Free-course registration separately relies on its unique learner–course constraint rather than this learner-lock workflow. Its pre-insert check can still race: one call succeeds and another can receive a failure even though the single enrolment is preserved.

Figure 9.2 shows the lifecycle seen by repeated requests. A duplicate does not reverse the learner or create a second protected reward; it returns the state that already won the race. This property matters on mobile networks, where the server may commit even when the client never receives the response.

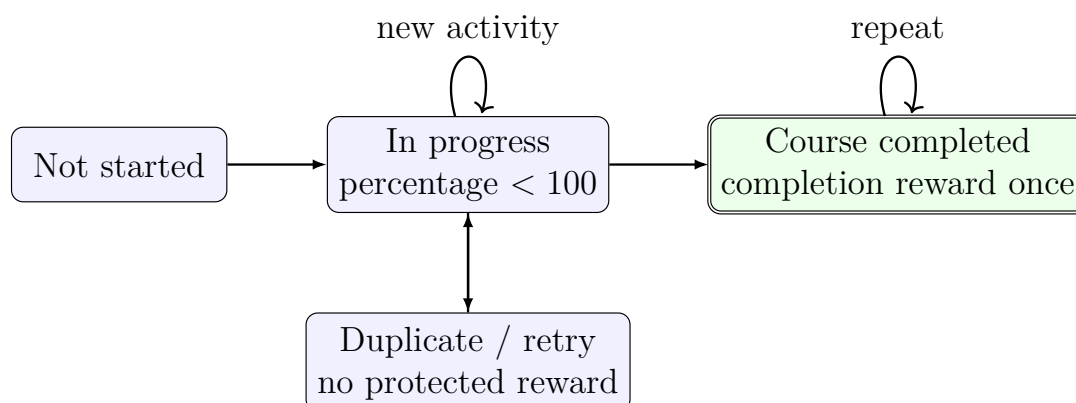


Figure 9.2: Learning-progress state and retry behavior

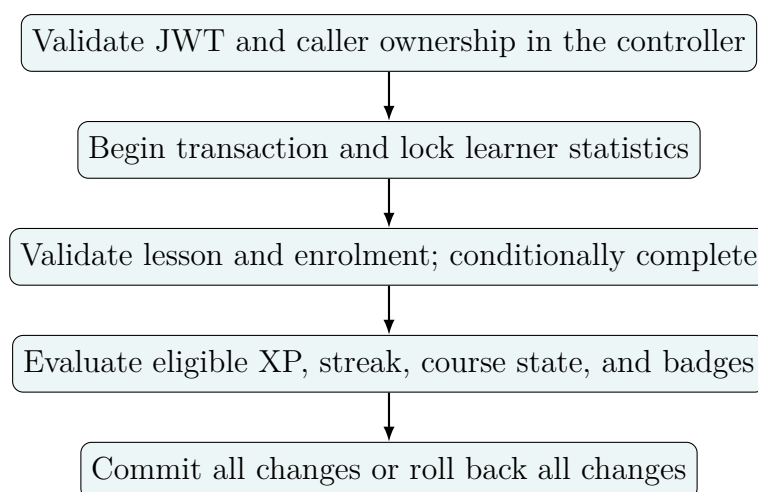


Figure 9.3: Content-completion transaction; multiple choice has additional attempt and quest writes

9.3 RabbitMQ embeddings

Course and lesson services publish a content-change event after transaction commit. A durable queue feeds a listener with prefetch 10 and three retry attempts. The listener creates a MiniLM vector, stores it, and invalidates related caches. Publication outcomes and processing duration are measured.

The configuration also declares a durable `gamification_queue` and its binding. No corresponding gamification listener implements the reward workflow in this snapshot. Rewards are computed synchronously inside PostgreSQL transactions. A declared queue is therefore not evidence of asynchronous gamification processing.

Figure 9.4 explains the ordering. Publishing before commit could let the consumer read old content or embed a write that later rolls back. After-commit publication avoids that ordering error and keeps model inference outside the HTTP response time.

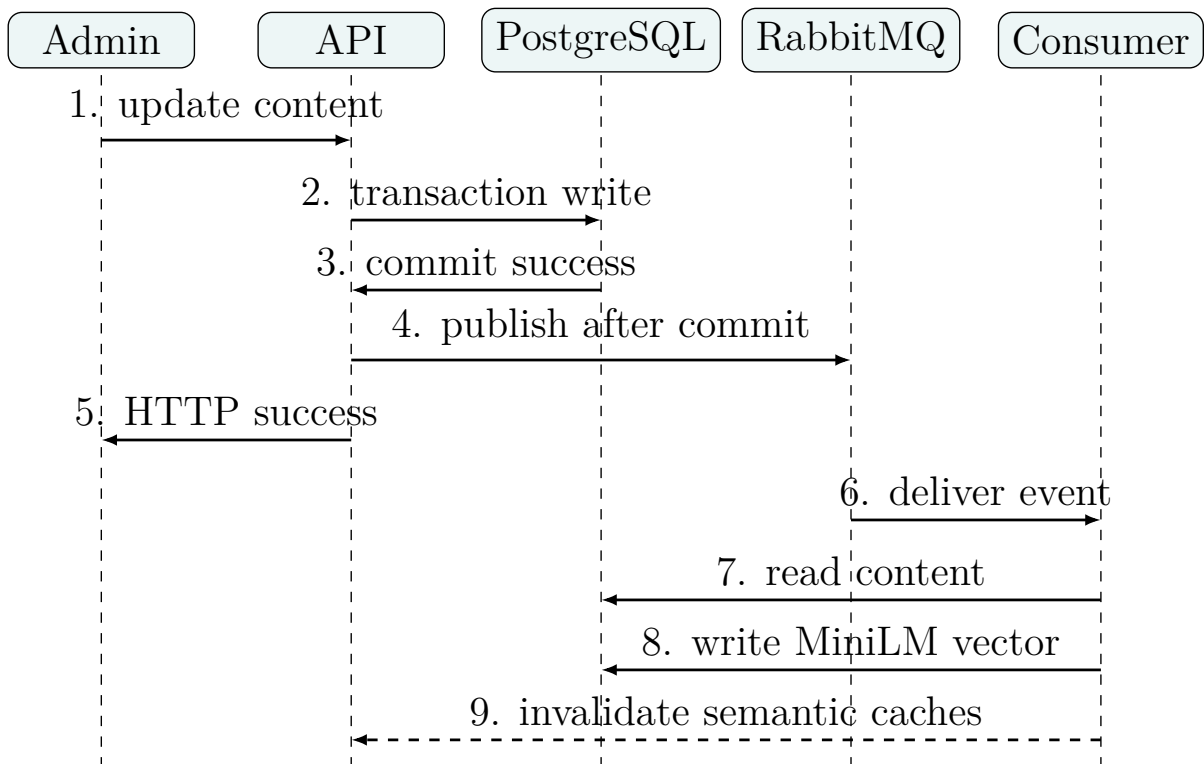


Figure 9.4: Post-commit content embedding sequence

The design currently has no transactional outbox or publisher confirmation. A committed content write can therefore exist without a durable event if publication fails. Operators must react to the alert and replay affected content.

9.3.1 Delivery semantics and stale vectors

The event payload contains a content type and UUID, and is routed through the `eduflex_exchange` direct exchange with `content.changed` to `content_embeddings_queue`. The

listener reloads current content instead of trusting a full text snapshot carried by the message. Reprocessing can overwrite an embedding without awarding a learner reward because the consumer owns derived vectors, not gamification.

Broker durability, publisher confirmation, and consumer acknowledgement are distinct boundaries [19]. Here, the *sent* metric means that the `RabbitTemplate` call returned; it is not a durable broker confirmation. Listener retries are bounded, but there is no explicit dead-letter/reconciliation workflow. A transaction outbox would close the database-commit-to-publication gap by committing the event record with the content and delivering it later.

Reloading current text avoids embedding an obsolete message snapshot, but it is not a full version-ordering protocol. With concurrent consumers, an older computation could still finish after a newer one and overwrite its vector. The current listener has no content-version comparison. Any future increase in worker concurrency should include versioned writes and an out-of-order completion test.

9.3.2 Calendar time and reward semantics

Daily rewards use the configured calendar zone, defaulting to `Asia/Ho_Chi_Minh`, rather than assuming every learner day ends at UTC midnight. The update awards ten XP only when the stored date is absent or earlier than today’s date. Login and explicit check-in share this boundary. Fixed-clock tests cover same-day retries, the next local day, and a future-dated record that must not move backward. Time is therefore an explicit input to the rule rather than a dependence on when the test happens to execute.

Daily check-in and study streak are separate fields and rules. The former records `last_login_xp_date`; it does not itself advance `last_study_date`. Streak updates set the count to one on first activity, preserve it on another activity on the same date, add one when the last date is yesterday, and restart at one after a gap. Unlike the check-in future-date guard, the streak method’s remaining branch also restarts a future-dated record. The report does not claim a shared future-date policy for both operations.

9.4 Worked progression examples

Let T be the number of lessons in a course with no parent, and let C be the number of those lessons completed by the learner. The stored display percentage is

$$P = \begin{cases} 0, & T = 0, \\ \text{round}(100C/T, 1), & T > 0. \end{cases}$$

Terminal course completion uses the integer condition $C = T$ with $T > 0$, rather than testing the rounded display value. This avoids treating a display rounded to 100 as proof that every counted lesson is complete.

Consider a course with two top-level content lessons A and B, and one quiz child Q under B. Here $T = 2$, not three. Completing A creates its progress row, gives 20 XP, and sets $P = 50$. Passing Q for the first time gives 30 XP and completes both Q and its parent B. The two top-level lessons are now complete, so the winning course transition gives 50 additional XP and awards the course badge. Ignoring independent daily and quest rewards, this ordering totals 100 XP. It does not grant the 20-XP content reward for B because B was completed by the quiz workflow.

If B was explicitly completed before Q, its first content transition earns its own 20 XP. Course completion may therefore occur earlier according to the top-level counting rule. This example shows why progress, assessment mastery, and reward eligibility should be specified separately: the current state machine is not a rule that every child quiz must be passed before a course can appear complete. Product requirements should decide whether that policy is appropriate before changing the denominator or introducing prerequisite enforcement.

Table 9.1: Source-derived ordering examples for A, B, and quiz child Q

Ordering	Learning XP	Reason
A, then Q	$20 + 30 + 50 = 100$	Q completes B; B's separate content reward is absent.
B, then Q, then A	$20 + 30 + 20 + 50 = 120$	Both content rewards precede their own completion boundaries; final A sees the prior quiz pass.
A, then B, then Q	$20 + 20 + 30 = 70$	Course becomes complete before any quiz pass; Q cannot win the already-completed course transition.

These source-derived cases exclude login and quest XP and are not new executed integration scenarios. They expose a policy gap beyond duplicate safety: at-most-once course reward does not prove that every eligible ordering receives it. Existing tests cover a final independent content lesson racing a quiz pass, rather than every ordering of the quiz's own parent completion. A future regression should exercise Table 9.1 against the agreed reward policy, potentially using a separate course-reward eligibility state instead of coupling eligibility to the first percentage-completion transition.

Table 9.2: Reward rules and their actual qualifying boundaries

Event	Current benefit	Qualifying boundary
Content completion	20 XP	New user–lesson completion transition
Multiple-choice pass	30 XP	No previous passing attempt for that quiz lesson
Course completion	50 XP through quiz; content path requires a prior course quiz pass	Winning incomplete-to-completed enrolment transition
Daily check-in/login	10 XP	New configured local date
Quest completion	Stored quest reward	New completion for user–quest–date
Level	$\lfloor XP/100 \rfloor + 1$	Resulting integer XP total
Static badges	First login, streak 7/30, XP 500/1,000	Named definition and singular user–badge award
Course badge	Dynamic course condition	Course completion; not educational accreditation

Failed and already-passed multiple-choice submissions still contribute to unfinished quiz-count quests. A perfect submission increments an unfinished perfect-run quest; a non-perfect submission resets that counter. A completed quest is not reset later that day. Thus the response’s `xpRewarded` field, which tracks first-pass and course XP, can be zero while `completedQuests` reports an independent quest reward. Clients must not assume the former is the complete XP delta for the request.

9.5 Known retry boundary

Quiz submissions create attempt records and do not accept a client idempotency key. A response-loss retry can create another attempt and increment an unfinished quest. First-pass XP, course XP, quest-completion XP, progress, and badges remain protected.

9.6 Failure analysis

Table 9.3: Reliability mechanisms and remaining boundaries

Failure	Current protection	Remaining action
Duplicate lesson request	Conditional transition and unique model	Return existing state without new XP.
Concurrent reward writes	Per-learner row lock and transaction	Observe lock time and pool pressure.
Exception after partial work	Transaction rollback	Retry the complete operation.
RabbitMQ publish failure	Content stays committed; failure metric increases	Replay content and add an outbox.
Consumer failure	Listener retries three times	Add explicit dead-letter handling.
Lost quiz response	Protected rewards remain single	Add an idempotency key for attempts.
Redis unavailable	Readiness and monitoring expose dependency failure	Restore Redis and define degraded behavior.

Correctness and throughput have different meanings. Serializing one learner’s reward mutations is intentional because those operations contend for one logical aggregate. Different learners lock different rows and can proceed independently. A load test is still required to measure connection-pool, database CPU, and lock-duration limits.

Chapter 10

AI-Assisted Learning

10.1 Retrieval

The local all-MiniLM-L6-v2 model produces 384-dimensional course and lesson vectors. A learner question is embedded and compared with lesson vectors from the selected course using pgvector. Constraining retrieval by course reduces unrelated context.

The embedding service uses LangChain4j’s in-process ONNX model rather than an external embedding API [12]. Its wrapper passes a text segment to the model and serializes the returned floats for PostgreSQL. The upstream model card describes an English sentence and short-paragraph encoder, with default truncation beyond 256 word pieces [22]. That model-card default should not be mistaken for a separately verified token limit of this Java wrapper. The deployed library’s tokenization and truncation need an explicit check before claiming that an entire long lesson is represented by its single vector. Vietnamese or mixed-language course retrieval also needs its own benchmark; local inference is not proof of multilingual retrieval quality.

Embedding is a representation step rather than an answer-generation step. Vector distance lets differently worded questions locate related lessons. The relational course filter defines retrieval scope. The controller supplies the enrolment or administrator check before retrieval; vector similarity supplies ranking rather than authorization.

10.1.1 Catalogue search and course question retrieval

Catalogue search has a different ranking policy from course question retrieval. The catalogue search service first requests up to ten keyword matches. If those results do not fill the limit, it embeds the query and adds semantic course matches. A LinkedHashMap preserves keyword-first order and removes duplicate course UUIDs. The final list is capped at ten; it is not a single globally reranked blend of lexical and vector scores. The cache key includes user identity because the query excludes courses in which that learner is already enrolled.

Course questions instead rank lesson vectors within one enrolled course. This separates discovery from grounded assistance: a good course-level title match does not prove that a particular lesson contains an answer. Query embedding remains synchronous in both search and question requests. RabbitMQ defers content-vector refresh; it does not move every AI operation off the request thread. Background inference also shares the API process’s CPU and memory, so queuing does not create independent compute capacity.

10.1.2 Scoring and context construction

The repository ranks lessons by pgvector cosine distance and returns relevance as $1 - d_{\text{cosine}}$. This is a similarity score, not a probability that the answer is correct. The query filters by course UUID, excludes null embeddings, and returns at most five lessons. When no vector result exists, the service selects up to five stored course lessons. That fallback is ordered by lesson UUID in the repository and is not a learned relevance ranking.

Listing 10.1: Course-scoped retrieval implemented by the repository

```
1 SELECT lesson_id, title, content,
2       (1 - (embedding <=> ?::vector))::float AS relevance
3 FROM lesson
4 WHERE course_id = ? AND embedding IS NOT NULL
5 ORDER BY embedding <=> ?::vector
6 LIMIT 5;
```

One vector is stored per lesson, not per independently indexed text chunk. Context construction truncates each lesson excerpt to 1,500 characters and uses a 14,000-character budget check for the excerpt accumulation. Metadata and headings mean this is not a strict token cap. The generator requests temperature 0.2 and at most 700 output tokens. These are code-level bounds; their adequacy for long lessons has not been measured. Versioned chunking and a labelled retrieval benchmark are appropriate extensions.

A useful future retrieval record would contain the content version, embedding model identifier, chunk boundaries, language, and generation timestamp. A model change cannot safely be treated as a cache refresh when vectors produced by different encoders share the same search space. The migration should regenerate a coherent index and compare ranked results before switching readers to it. These metadata and version-switching mechanisms are proposed; the current rows hold only the lesson or course vector alongside their other content fields.

10.2 Grounded generation and fallback

Retrieved lesson excerpts become context for optional Gemini generation. The result contains source information so the UI can identify supporting material. Expensive summaries

and search results are cached. Without `GEMINI_API_KEY`, summaries use a deterministic overview and questions return retrieved lesson suggestions, allowing local development and tests without provider access.

The inspected default model remains `gemini-2.0-flash`. Google’s release notes record that model’s shutdown on 1 June 2026 [5]. Consequently, configuring only an API key is not evidence that the default live-generation path works at this report’s October snapshot. A supported model must be selected through `GEMINI_MODEL` and verified against the integration’s request/response contract. No credentialed provider check was performed during this report revision. Existing fallback behavior is useful degradation, but it does not repair an obsolete model configuration or establish generated-answer quality.

The backend returns sources with the answer instead of presenting generated prose alone. This gives the learner a route back to the lesson and lets an evaluator inspect retrieval quality. If retrieval selects weak evidence, fluent wording does not make the evidence stronger; source visibility makes the weakness observable.

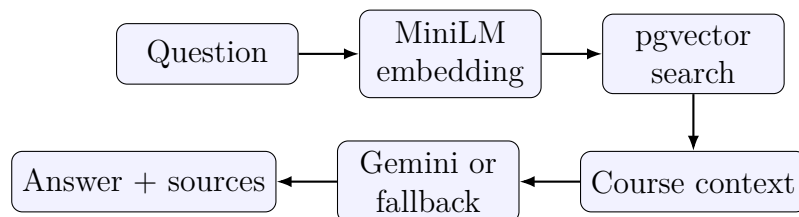


Figure 10.1: Grounded course-question flow

10.3 Responsible boundaries

Prompts, learner IDs, emails, tokens, and questions are excluded from metric labels. Answers remain attributable to stored course content and are not treated as authoritative academic assessment. Provider outcomes use bounded metric labels.

Course text is embedded locally, but retrieved excerpts and the learner’s question are sent to Gemini when generation is enabled. Local embeddings therefore do not mean that the entire AI path remains on the host. An instructor-uploaded excerpt is also untrusted prompt input: an instruction in course material should not override application policy. Source attribution and a low generation temperature are useful presentation choices, but they are not prompt-injection controls or guarantees against hallucination.

10.4 Quality evaluation approach

The current automated test proves deterministic fallback behavior. A broader AI evaluation should prepare representative questions for each course, expected lesson sources, and

educator-reviewed answer criteria. Retrieval can then be measured through source recall and ranking, while generated responses can be judged for groundedness, completeness, and unsupported claims. Latency and provider failure rate should be reported separately from content quality.

This evaluation remains future work because the repository does not contain a labelled educational benchmark. The report describes the implemented retrieval pipeline without claiming a measured pedagogical quality threshold.

Chapter 11

DevOps, Deployment, and Monitoring

11.1 Containerization and configuration

Docker Compose builds the API and runs PostgreSQL/pgvector, Redis with append-only persistence, and RabbitMQ. Health checks guard dependency readiness. Example environment files document variable names; real credentials stay untracked.

11.2 Continuous integration

GitHub Actions runs three dependent jobs. The backend job uses JDK 21 and executes Maven/Testcontainers tests. The monitoring job starts an isolated stack and validates it. The Android job installs SDK 35, runs lint, builds debug and release APKs, and builds a release AAB. Reports and artifacts are retained for 14 days.



Figure 11.1: Continuous integration order

11.3 Metrics, dashboards, and alerts

Actuator and Micrometer expose JVM, HTTP, HikariCP, and custom metrics on an internal management listener. Prometheus scrapes the API, its main-listener probe, PostgreSQL, Redis, and RabbitMQ exporters [18]. Custom metrics cover progress, rewards, statistics-lock duration, embedding publication and processing, and AI outcomes.

Grafana provisions Service Overview, Dependencies, and Learning Operations dashboards [9]. Alerts cover API availability, absent targets, 5xx rate, p95 latency, pool con-

tention, queue backlog, absent consumers, publication failure, RabbitMQ alarms, Redis evictions, and PostgreSQL deadlocks. Alertmanager groups alerts locally.

Figure 11.2 separates telemetry sources from storage, display, and local alert inspection. A healthy Actuator scrape proves that the management listener is alive; the Blackbox probe separately tests the client-facing API port. The two signals prevent a healthy metrics port from hiding a failed application listener.

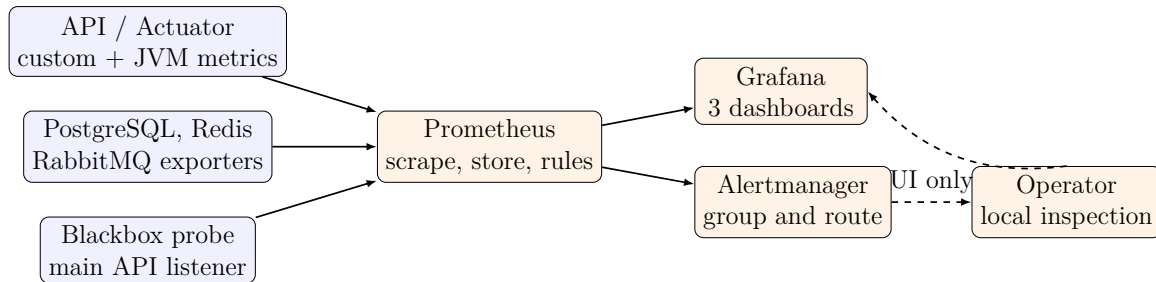


Figure 11.2: Metrics, visualization, and alerting flow

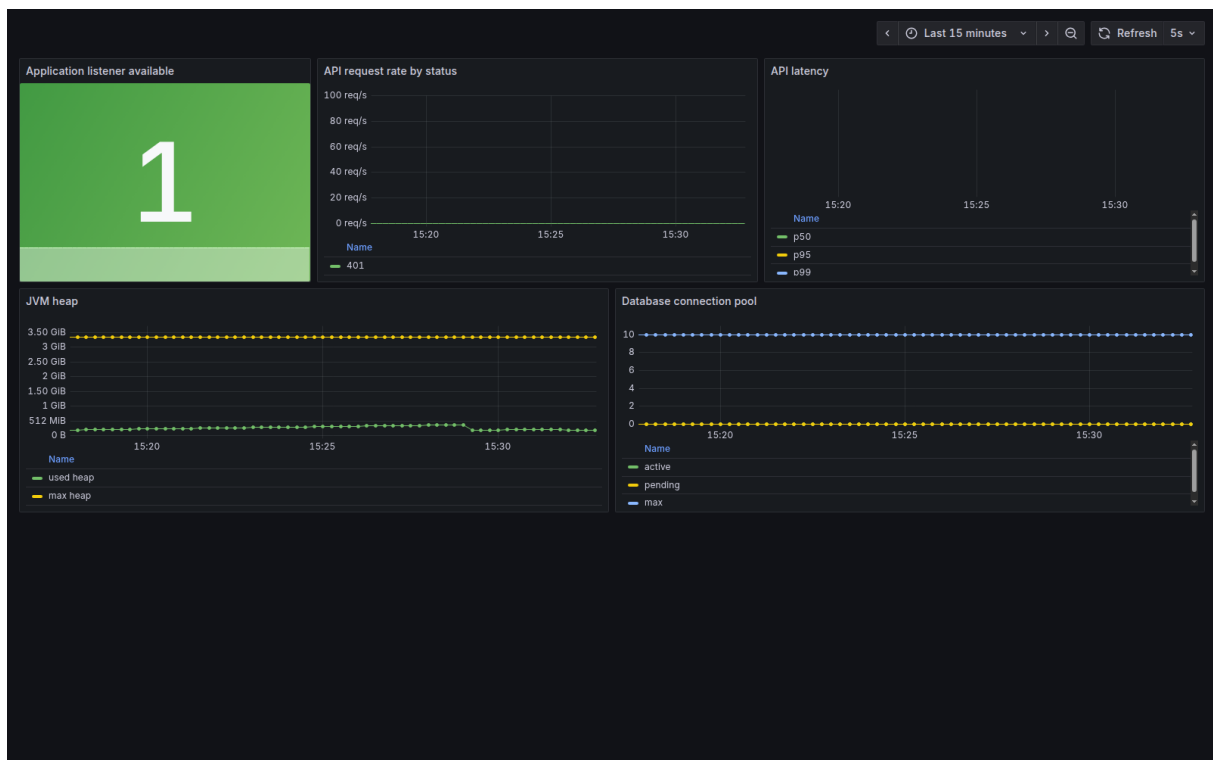


Figure 11.3: Provisioned Grafana Service Overview running against the local stack

Figure 11.3 is a real local capture rather than a mock-up. It shows the independent application-listener probe, request status, latency panels, JVM heap, and Hikari connection-pool capacity. The near-idle request and latency panels reflect the capture workload; they must not be interpreted as a benchmark.

11.4 Operational security

11.4.1 What custom metrics do and do not establish

Progress and reward counters register their increments after transaction commit, so an injected rollback does not appear as a committed reward. They remain operational counters: process restarts reset them, and they are not a durable accounting ledger. The statistics acquisition timer wraps both row initialization and the locking query; it includes database round-trip work and is not an isolated measurement of lock wait. Correlating it with Hikari pending connections and PostgreSQL activity is therefore more useful than assigning every increase to one cause.

AI counters distinguish provider output from fallback by mode. The fallback label includes both absent configuration and provider failure, so it cannot by itself establish a provider-error rate. Similarly, an embedding processing success measures the listener operation, not necessarily the relevance of the stored vector. These definitions prevent visually impressive dashboards from overstating evidence.

11.4.2 Exposure and access

Liveness and readiness are exposed on the main API port. The monitoring profile exposes only health and Prometheus on internal port 8081. Grafana, Prometheus, and Alertmanager bind to loopback locally. The PostgreSQL exporter uses a dedicated `eduflex_monitor` role with `pg_monitor`. Prometheus retention is bounded to seven days or 2 GB.

The current Alertmanager receiver groups alerts for inspection in its local web UI but does not send email, chat, or paging notifications. External delivery is therefore a future deployment task rather than an implemented operational guarantee.

11.5 Operational interpretation

The dashboards correspond to questions asked during an incident. Service Overview shows whether the API is unreachable, slow, failing, or waiting for database connections. Dependencies narrows pressure to PostgreSQL, Redis, or RabbitMQ. Learning Operations shows progress transitions, rewards, row-lock time, embedding work, and AI outcomes.

An alert starts an investigation; it does not prove one cause. High latency together with Hikari pending requests points toward database work or pool saturation. A growing embedding queue with zero consumers points toward listener startup or broker connectivity. Redis eviction requires inspecting memory policy and workload before capacity changes. The monitoring runbook records these decision paths and recovery checks.

11.5.1 Measured signals and configured thresholds

An availability signal describes an observed condition; an alert threshold describes the operator’s configured decision policy. The current one-second latency threshold is not a measured service-level objective. Request-volume guards reduce noisy ratio alerts during very light traffic, which is particularly relevant to a local student project.

Table 11.1: Selected implemented alert conditions

Signal	Threshold and duration	Interpretation
Client-facing API probe	Probe fails continuously for one minute	API listener or readiness dependency unavailable.
HTTP server errors	More than 5% 5xx with at least 100 requests in five minutes; holds for five minutes	Investigate the failing route and dependency, not ordinary 4xx.
API latency	Estimated p95 exceeds one second with the same volume guard; holds for five minutes	Inspect pool waiting, database work, JVM, and expensive AI routes.
Hikari pending requests	Pending count remains above zero for two minutes	Connections are occupied or waiting; pool growth alone may worsen pressure.
Embedding backlog	More than 100 ready messages for five minutes	Arrival rate exceeds processing capacity or processing has stalled.
Embedding worker absent	Ready messages and zero consumers for two minutes	Listener is disconnected or failed to start.
Publish failure	Failed publication counter increases within five minutes	Committed content may lack a refreshed vector.

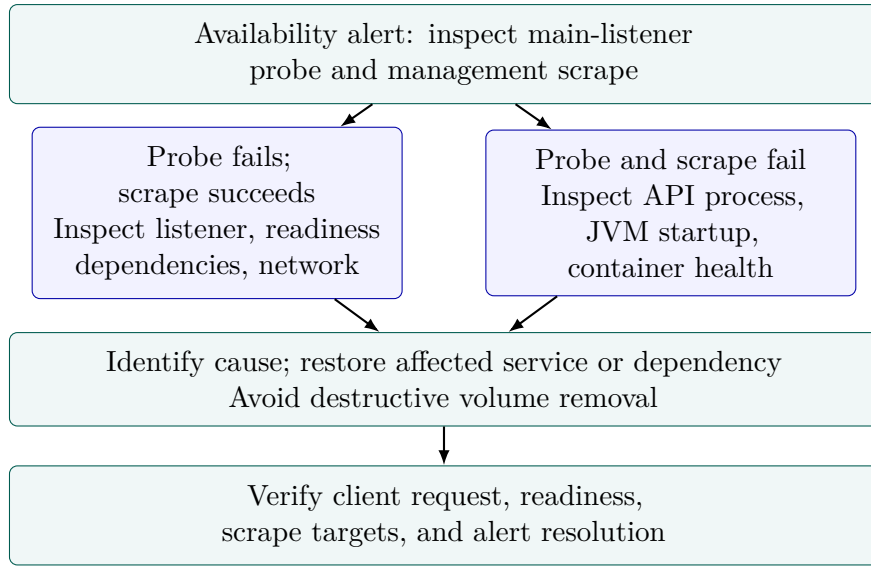


Figure 11.4: Runbook-based investigation and recovery decision model

Figure 11.4 is an operator investigation model built from the versioned runbook. Checking the main listener and management scrape separately narrows the failure boundary before restarting anything. Recovery is complete only when traffic, dependency health, scrape targets, and alert resolution agree.

11.5.2 Operational maturity and deployment limits

The local topology has one database instance, one broker, and one API process. It does not provide automatic failover, database point-in-time recovery, an ingress TLS layer, or a rolling-deployment controller. Redis AOF and named database volumes preserve local state through ordinary restarts, but they are not backups. An operator must separately test restoration from an independent copy.

Refresh-token revocation currently finds a user’s Redis keys by pattern. This is simple for a small environment but should be replaced by a bounded scan or indexed session set before a large keyspace is supported. Separating session state from disposable caches would also make eviction policy easier to reason about. These limits explain why passing the monitoring smoke test does not establish production readiness.

11.6 Recovery design and state classification

Recoverability depends on which state was lost. PostgreSQL holds identities, enrolments, attempts, progress, and reward totals. Redis holds both disposable read caches and security-relevant refresh-token records. RabbitMQ carries unfinished embedding work; a vector can be regenerated from current content, but a missing reward history cannot be reconstructed from a cache. This classification determines the recovery order instead of

treating every container as interchangeable.

Table 11.2: Recovery requirements by stored state

State	Current preservation	Recovery requirement
PostgreSQL	Named local volume	Independent backup, extension availability, roles, schema version, restored invariant checks
Redis cache	AOF on named volume; values have TTLs	Safely invalidate derived values; do not serve pre-restore learner data
Refresh sessions	Stored in the same Redis service	Explicit session invalidation decision; database-only restore does not revoke issued access tokens
Embedding messages	Durable queue; no broker	Reconciliation and auditable replay of missing work
Embeddings	Vectors reside in PostgreSQL	Match content/model versions or regenerate through future tooling
Media	External storage objects referenced by URL	Verify independent provider backup and access; database restore does not restore remote objects
Monitoring	Provisioned configuration and local data volumes	Restore configuration first; retained metrics do not restore application state

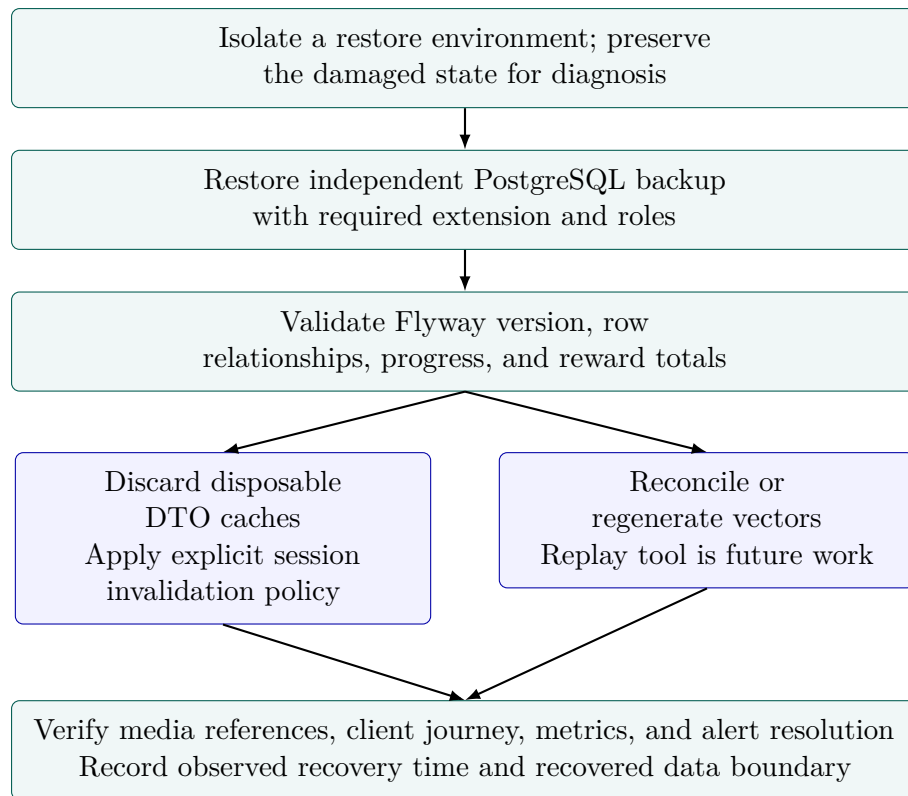


Figure 11.5: Proposed isolated restoration drill; backup automation and replay are not implemented

Figure 11.5 is a proposed exercise, not evidence of an executed recovery. A suitable first step is a PostgreSQL logical backup restored into an isolated compatible database. PostgreSQL documents the snapshot consistency of `pg_dump`, custom-format restoration with `pg_restore`, and the separate handling of cluster roles [15]. The project does not yet automate these procedures. A continuous WAL archive and base backup would be a different recovery design; the current Compose volume does not supply it.

The drill should state the latest recoverable data boundary and elapsed time until verified service restoration, rather than inventing a recovery-time target. It should compare representative user, enrolment, progress, attempt, and XP records before and after restoration. It should also test whether stale caches or still-issued tokens can expose a database state that has moved backward. Resetting a container without restoring an independent copy is a restart test, not a backup-restoration test.

11.6.1 Reproducibility and deployment drift

The Compose files pin several monitoring images to explicit versions, while `pgvector/pgvector:pg16`, `redis:alpine`, and `rabbitmq:3-management-alpine` are floating tags. They identify a product or release family rather than immutable image bytes. An environment manifest should record resolved digests, JDK, schema version, active profile, dependency versions, and non-secret configuration alongside the source revision. Wrap-

pers and Flyway improve repeatability, but they do not make mutable image tags or an uncommitted working tree immutable.

Before a deployment, migrations should be rehearsed on a representative restored dataset, health checks should pass on both the application and management listeners, and the client contract should be checked against the active API. Adding API replicas would introduce additional cache and embedding races rather than automatically proving horizontal scalability. The report’s single-host topology is the inspected implementation; a multi-host deployment requires its own design and evidence.

Chapter 12

Testing and Evaluation

12.1 Strategy

Verification combines unit tests, database integration tests, live API journeys, Android emulator checks, lint, builds, and monitoring smoke tests. Attention focuses on token semantics, rollback, ownership, duplicate rewards, concurrency, and startup.

The report separates fresh evidence from historical checks. The backend was rerun on 4 October 2026 with JDK 21 and Docker against the working tree based on `3a71b37`, including uncommitted application changes. A commit identifier alone does not reproduce that working tree. The new run is summarized in `report/verification.md`. Android unit-test XML records are dated 3 October 2026.

The earlier live HTTP, emulator, Android release-build, and monitoring results come from 24–25 September 2026 and are retained as historical evidence, not described as fresh passes for the current code. Their records are `docs/reliable-progress-verification.md` and `docs/monitoring-runbook.md`. This distinction matters because the payment and recovery contracts have evolved.

No single test layer proves the whole system. Unit tests give fast feedback for token and fallback behavior. Testcontainers supplies real transaction, constraint, and lock semantics. The live API journey checks security filters, serialization, persistence, and messaging through HTTP. The emulator adds Android navigation and contract mapping. Monitoring verification confirms that operational signals remain usable after startup.

Table 12.1: Evidence mapped to quality goals

Goal	Evidence	What it establishes
Security	JWT tests and rejected HTTP flows	Token type, role, and ownership in tested routes.
Consistency	PostgreSQL concurrency suite	Single protected transitions under duplicate calls.
Atomicity	Injected transaction failure	Related progress and rewards roll back together.
Integration	26-check live HTTP journey	Main backend workflow across real dependencies.
Mobile compatibility	Contract test and API 35 emulator	Token mapping and representative navigation.
Delivery	Lint, APK, and AAB builds	Source and shrinking configuration compile.
Observability	Targets, dashboards, and alert drill	Metrics and alert lifecycle work locally.

Table 12.2: Claim-to-evidence traceability

Report claim	Repository evidence	Boundary
Protected rewards are single under duplicate concurrency	Progress concurrency integration test; final database assertions after 100 synchronized calls	Correctness for defined scenarios, not capacity
Related writes are atomic	Injected exception and retry integration scenario	One transaction path and PostgreSQL configuration
Authorization rejects wrong ownership	Integration scenarios plus live HTTP journey	Exercised routes, not a complete penetration test
Embedding crosses RabbitMQ	HTTP journey waits for stored vector after content change	Successful delivery path; no outbox guarantee
Refresh revocation works	JWT unit tests and refresh/logout HTTP checks	Refresh token only; issued access token remains valid until expiry

Report claim	Repository evidence	Boundary
Android consumes the contract	Login response unit test and API 35 emulator journey	Representative navigation, not full device coverage
Monitoring detects API loss	Versioned rules, seven targets, and stop/recovery drill	Local Compose; receiver has no external notification route
Release source compiles with shrinking	Gradle release APK/AAB tasks	Unsigned inspection artifacts

12.2 Backend results

The fresh run used Java 21, Docker Engine 29.8.1, and PostgreSQL 16/pgvector. All 28 migrations ran. The result was 28 tests with zero failures, errors, or skips; 23 scenarios belong to the progress concurrency integration suite. Three password-recovery unit tests and one test each for JWT semantics and AI fallback make up the remaining five.

Table 12.3: Selected verified scenarios

Scenario	Observed result
100 concurrent completions of one lesson	One progress row and exactly 20 XP.
100 concurrent daily check-ins	One date transition and exactly 10 XP.
100 concurrent first statistics reads	One row without uniqueness failure.
Repeated lesson completion	Timestamp and XP preserved.
Different lessons completed concurrently	Both rewards and percentage correct.
Concurrent first quiz passes	First-pass and course XP awarded once.
Two learners complete one course	One badge definition and one award per learner.
Injected failure after related writes	Full rollback; retry succeeds once.

Scenario	Observed result
Unauthorized and wrong-owner calls	Rejected without state mutation.
Daily boundary with fixed clock	Same-day, next-day, zone, and future cases pass.
Profile update for another learner	Rejected with 403.
AI assistance without enrolment	Rejected; course access checked before retrieval.
Paid course through free enrolment	Rejected; repeated enabled simulation creates one successful payment and enrolment.
Payment for another learner	Rejected without granting course access.
Reset code replay or expiry	Consumed or expired code does not authorize another reset.

These tests establish correctness for the tested cases; they are not a throughput benchmark and do not claim production latency.

The concurrency harness starts 100 worker threads together and permits up to 32 simultaneous test database transactions. Assertions inspect final database state, not only successful responses. This matters because every call could return success while still duplicating XP internally. A pinned PostgreSQL/pgvector image and Flyway startup keep tested database behavior close to the local runtime.

12.3 End-to-end and Android results

The historical live HTTP journey completed 26 checks with no failures. It covered administrator and learner registration, protected content creation, catalogue reads, cross-user rejection, enrolment, progress, quiz retry, XP totals, embedding completion, refresh, logout, and revoked-token rejection.

The debug APK was exercised on an Android 15 API 35 emulator. A learner registered, logged in, opened the main activity, accepted notification permission, viewed Home, opened Learn, and received successful authenticated statistics, quest, and enrolment responses. A unit contract test protects access-token response mapping.

The journey also waited for RabbitMQ embedding completion, demonstrating that a

content update can travel through broker and consumer into vector storage. It checked token refresh followed by logout and rejection of the revoked token. These steps join otherwise isolated features into a meaningful lifecycle.

12.4 Build and monitoring results

Android debug lint, debug APK, release APK, and release AAB tasks succeeded with release shrinking enabled. Monitoring verification checked Compose configuration, alert rules, metric protection, seven Prometheus jobs, and three dashboards. A controlled API stop caused the availability alert to fire after one minute and resolve after recovery.

The drill checks both directions of the alert lifecycle. Firing demonstrates detection; resolution confirms that the rule, scrape path, and restored service agree after recovery instead of leaving a stale incident.

12.5 Evaluation limits

12.5.1 Answers to the evaluation questions

Table 12.4: Evaluation questions answered within the available evidence

Question	Answer supported by evidence	Qualification
Q1: journey	Historical HTTP checks cover the main learning lifecycle; emulator evidence covers registration, login, Home, and Learn	A complete current Android course-to-certificate journey remains unverified.
Q2: consistency	Fresh PostgreSQL tests preserve singular protected rewards, timestamps, ownership, and rollback in the defined scenarios	Quiz attempts may still duplicate; the test is not a throughput benchmark.
Q3: derived work	Source inspection and historical delivery checks show embedding inference outside the content transaction and versioned operational signals	Missing outbox, cache race tests, and broker persistence prevent a universal recovery guarantee.

Question	Answer supported by evidence	Qualification
Q4: reproducibility	Fresh backend run applies the versioned schema and executes every current test; report source is self-contained	Current application tree includes uncommitted changes; other build and monitoring evidence is historical.

12.5.2 Threats to validity

The clean Testcontainers database controls schema setup and makes transaction tests repeatable, but does not reproduce a large upgraded dataset or prolonged production traffic. The concurrency harness synchronizes service callers and verifies final state; it does not reproduce arbitrary mobile network delay, every transaction isolation setting, or independent API instances. Test configuration also differs from runtime configuration in its pool size and external-service substitutes.

The historical emulator evidence covers one API level and representative navigation. It cannot establish accessibility, usability, or device diversity. AI fallback tests validate deterministic behavior rather than educator-approved answers. These are limits of coverage and experimental design, rather than reasons to discard the verified database invariants.

12.5.3 Source audit findings outside the passing suite

The report review found gaps that the current passing tests do not establish as correct. Table 12.5 records source-derived findings and the next test needed to validate a correction. These are findings about the application snapshot, not claims that this report revision has changed the application.

Table 12.5: Uncovered implementation gaps and required regression evidence

Finding	Practical effect	Required correction evidence
Submitted-list quiz denominator	A subset or repeated question may receive a passing score	Reject missing, duplicate, foreign, and null answer items; grade a complete server-owned set

Finding	Practical effect	Required correction evidence
Fill-blank lacks shared scope and persistence	Feedback does not establish enrolled access or progression; missing key can fail	Wrong-course and non-enrolled requests rejected; defined practice/progress semantics; missing data returns a controlled error
Client-reported quest increment	An owner can claim unobserved activity toward rewards	Only approved server events affect quiz quests; replayed events have one contribution; bounded time policy
Review eligibility differs by route	The same storage has two enrolment policies	Equivalent accepted/rejected outcomes for canonical and compatibility routes
Content reads lack course entitlement checks	Authentication alone does not enforce paid-course content access	Enrolled, non-enrolled, and administrator read matrix
Progress counts parents, not every child quiz	Display completion is not proof of full assessment mastery	Product-approved required-unit model and ordering tests
Parent completion can precede quiz eligibility	A course bonus can be omitted when completion wins before the first quiz pass	Exercise all A/B/Q orderings and verify the agreed total with an explicit reward-eligibility model
Default Gemini model is retired	API-key configuration alone does not restore live generation	Select a supported model and verify a credentialed response, fallback, and source contract
Eviction omits outer courses and newly created lesson lists	A successful mutation may remain invisible until cached data expires	Warm route, mutate, immediately reread; assert updated state for every dependent representation

Finding	Practical effect	Required correction evidence
Broad catches in transactional content creation	Failure DTO does not independently prove rollback	Inject failure between parent and child creation; verify neither persists
No representative populated-schema upgrade	Clean migration success misses data transformations	Restore older fixtures, migrate, compare relationships and retained successful payments

The current concurrency fixtures use known valid answer sets and verify reward transitions. That makes them useful for transaction correctness while leaving assessment completeness untested. A convincing evaluation should separate three questions: whether an input is permitted, whether its business result is correct, and whether concurrent execution preserves that result. A test count is a coverage inventory, not a substitute for these distinct assertions.

12.5.4 Unmeasured product and performance outcomes

Emulator checks do not cover every device, screen size, or accessibility setting. No formal usability study or load-test dataset has been completed. External providers require separate credentials. These limits prevent unsupported claims and define future evaluation work.

A formal next evaluation should record task completion time and error rate for new learners, accessibility findings, p50/p95/p99 latency under a defined workload, database and queue saturation points, and recovery time after dependency failures. Each result should state dataset size, hardware, concurrency, cache state, and the script used to reproduce it.

12.6 Performance claims and missing measurements

Redis and RabbitMQ change where work occurs, but their presence alone does not prove a performance improvement. The project has not yet recorded uncached versus cached latency, synchronous versus queued embedding response time, sustained throughput, or resource saturation. Consequently, this report describes the intended performance mechanism and does not attach numerical speed-up claims to it.

Table 12.6: Required performance experiment design

Question	Controlled comparison	Reportable measures
Does caching help?	Cold/miss, warm/hit, and post-eviction reads with the same dataset	p50/p95/p99 latency, database calls, hit ratio
Does queuing shorten writes?	Content update with synchronous embedding baseline versus post-commit publication	HTTP latency, queue delay, end-to-end embedding time
Where does concurrency saturate?	Increasing learners and requests with fixed hardware and pool settings	Throughput, errors, lock time, CPU, pool pending
How does failure recover?	Stop and restore Redis, RabbitMQ, API, and consumer independently	Detection time, recovery time, lost/replayed work

12.6.1 A repeatable measurement protocol

The following protocol is proposed, not a completed benchmark. First, seed a fixed dataset with documented numbers of users, courses, lessons, enrolments, and vectors. Keep the same data, API profile, database pool, image digests, machine resources, and test script for each comparison. Disable external generation for core API measurements and evaluate provider-backed requests separately, so remote provider variation is not mistaken for database or cache behavior.

Measure catalogue reads, progress mutations, and course questions as separate workloads. For cache experiments, use cold, warm, and post-mutation phases. Confirm the expected cache outcome and returned content in each phase; warm reads with stale data are not a successful optimization. For concurrency, compare many callers for one learner with the same request count spread across distinct learners. These exercise deliberate per-learner serialization and shared resource saturation respectively. Include new completions and retries separately because their database work and reward outcomes differ.

Record the offered request rate, completed throughput, client-observed latency, timeouts, HTTP outcomes, and correctness assertions. Report percentiles per route rather than only a pooled average. Count timed-out and failed requests in the experiment outcome instead of silently excluding them. Repeat steady-state runs, publish the raw records, and preserve the warm-up and measurement windows in the script. A closed-loop caller count must be described as such; it does not represent an independently scheduled arrival

rate.

For embeddings, record both content commit time and the time the matching current vector becomes available. Consumer execution duration alone excludes time waiting in the queue. The existing metrics expose backlog and processing duration, but not an explicit event-age measurement. A timestamp/version signal and a reconciliation assertion are needed to measure end-to-end freshness. This separates the shorter HTTP write path from the eventual availability of updated search results.

The published result should include a configuration manifest, dataset seed, workload script, raw request records, final invariant checks, and an interpretation of the first saturated resource. Configured alert thresholds may help interpret the run, but they should not be retroactively presented as a measured service-level objective.

Chapter 13

Conclusion and Future Work

13.1 Conclusion

EduFlex delivers a mobile learning experience supported by a secure, transactional, and observable backend. It covers the path from Material-based Android screens to REST services, relational storage, caches, messaging, semantic retrieval, integration tests, containers, and dashboards.

The central achievement is trustworthy learning state. Progress and gamification are coordinated through transactions, row locks, conditional transitions, and constraints. Tests demonstrate that concurrent duplicate calls do not duplicate protected rewards. Caching and asynchronous embedding improve the design without moving authority away from PostgreSQL.

Observability is part of the implementation: metrics represent infrastructure and learning operations, dashboards are version-controlled, and availability alerting has been exercised. CI combines backend correctness, monitoring, and Android delivery.

13.2 Future work

1. validate complete assessment submissions, define fill-blank progression, derive reward activity on the server, and align content/review entitlement rules;
2. add a transactional outbox, publisher confirmation, and reconciliation;
3. add client idempotency keys and a reward ledger for quiz retries, and replace payment simulation with verified provider settlement if commercial checkout is required;
4. add structured centralized logs and OpenTelemetry traces;
5. configure external alert delivery and encrypted backups with restore drills;
6. load-test catalogue, progress, and AI paths and define measured objectives;
7. expand Android instrumentation, accessibility, offline, and device testing;

8. implement secure release signing and an approved store workflow; and
9. evaluate AI answers with a labelled dataset and educator review.

13.3 Prioritized engineering roadmap

The next increments should close known correctness and recovery gaps before adding more visible features. The roadmap distinguishes implemented mechanisms from proposed work and names an acceptance condition for each increment.

Table 13.1: Future increments and required proof

Priority	Change and justification	Acceptance evidence
P1	Assessment scope, full question coverage, and consistent quiz semantics	Missing, duplicate, foreign, and unknown questions rejected; valid complete sets scored against the server definition.
P1	Trusted activity and canonical entitlement rules	Arbitrary client increments cannot earn quiz rewards; both review routes and content reads follow the approved policy.
P1	Supported optional generation model and bounded provider calls	Verify credentialed request compatibility, timeout recovery, fallback, and returned source metadata.
P1	Outbox, broker persistence, confirmations, and dead-letter/replay tooling	Commit during broker outage; restore it; observe one current vector and an auditable replay trail.
P1	Quiz request identity and durable result storage	Repeat one client key after response loss; preserve one attempt and one quest contribution.
P1	Recovery abuse controls and explicit cross-system failure handling	Test expired/reused codes, concurrent issuance, SMTP failure, and Redis failure without unauthorized reset.

Priority	Change and justification	Acceptance evidence
P2	Load tests and route-specific objectives	Publish workload, cache state, hardware, throughput, latency percentiles, and saturation behavior.
P2	Android device and accessibility evaluation	Record task success, recovery behavior, large-font rendering, and TalkBack findings.
P2	AI chunk/version lifecycle and groundedness benchmark	Report source recall, unsupported claims, provider failures, and long-lesson retrieval quality.
P3	External alerts, backup restoration, TLS, and signed delivery	Demonstrate notification receipt, recovery from a backup, protected ingress, and a verified signed release.

13.4 Portfolio summary

Designed and implemented a native Android learning platform with a Spring Boot API, PostgreSQL/pgvector, Redis, RabbitMQ, JWT security, grounded AI assistance, and Prometheus with Grafana. Built concurrency-safe progress and reward transactions and verified single-reward behavior under 100 simultaneous duplicate requests with Testcontainers.

Appendix A

Build and Verification Commands

Listing A.1: Start the base environment

```
1 cp .env.example .env
2 cp backend/.env.example backend/.env
3 docker compose --env-file backend/.env \
4   -f backend/docker-compose.yml up --build -d
```

Listing A.2: Run backend tests

```
1 cd backend
2 ./mvnw --batch-mode test
```

Listing A.3: Run Android verification

```
1 ./gradlew --no-daemon \
2   :app:lintDebug :app:assembleDebug \
3   :app:assembleRelease :app:bundleRelease
```

Listing A.4: Verify monitoring

```
1 docker compose --env-file backend/.env \
2   -f backend/docker-compose.yml \
3   -f backend/docker-compose.monitoring.yml up --build -d
4 bash backend/scripts/verify-monitoring.sh
```

Appendix B

Selected API Groups

Route group	Responsibility
/api/user	Registration, login, recovery, and profile update.
/api/auth	Token refresh and logout.
/api/course	Catalogue, search, reviews, summary, and questions.
/api/lesson	Lesson creation and retrieval.
/api/enrollment	Learner enrolment and reads.
/api/progress	Lesson completion and course progress.
/api/quiz	Question retrieval and submissions.
/api/payment	Payment record processing.
/api/users/...	Statistics, streaks, quests, badges, and leaderboard.
/api/media	Avatar, image, and video upload.
/api/admin	Protected user and content administration.

B.1 Methods, identity, and policy differences

Table B.2 expands the groups into selected concrete methods. “Owner” means comparison with the verified JWT subject. “Authenticated” means that a bearer access token is required, without implying course entitlement. Public session routes still validate the credentials or token submitted in their payload. These distinctions describe the inspected source, including known gaps.

Table B.2: Selected route access and contract characteristics

Method and route	Identity and behavior
POST /api/user/register , /api/user/login	Public JSON account/session requests; validated DTOs
POST /api/user/forgot-password , /api/user/reset-password	Public; configured delivery and single-use reset code
POST /api/auth/refresh , /api/auth/logout	Public route policy; supplied refresh token determines session operation
PUT /api/user/update-profile/{userID}	Owner; validated JSON profile update
GET /api/course , /api/course/search	Authenticated catalogue reads; search uses caller identity
GET /api/course/my-courses	JWT identity determines learner's list
POST /api/course	ADMIN content creation
GET /api/lesson?courseID=...	Authenticated; no enrolment check in inspected read path
POST /api/lesson	ADMIN; model-bound lesson request
GET /api/quiz/{lessonId}	Authenticated; no enrolment check in inspected read path
POST /api/quiz/submit-multiple-choice	Owner plus enrolled quiz lesson; JSON; transactional result
POST /api/quiz/fill-blank	Owner; JSON feedback; no equivalent scope/progression workflow
POST /api/enrollment/{courseId}/register	Owner; JSON; paid courses reject free enrolment
GET /api/enrollment/{userId}	Owner enrolment list
POST /api/progress/lesson	Owner plus enrolment; form-encoded completion
GET /api/progress/course/{courseId}/user/{userId}	Owner course progress read
POST /api/payment , /api/course/payment	Owner; shared development simulation; server-side amount
GET /api/course/{courseId}/ai-summary ; POST /api/course/{courseId}/ask	Enrolled learner or ADMIN; optional generation with fallback

Method and route	Identity and behavior
POST <code>/api/course/reviews</code>	Owner from payload; enrolment required
POST <code>/api/course/{courseId}/reviews</code>	JWT identity; current use case omits enrolment rule
POST <code>/api/users/{userId}/daily-quests/progress</code>	Owner; client-reported quest type and increment
POST <code>/api/users/{userId}/xp</code>	ADMIN; explicit XP grant
POST <code>/api/users/{userId}/daily-checkin,</code> <code>/api/users/{userId}/streak</code>	Owner; separate calendar rules
POST <code>/api/media/users/{userId}/avatar</code>	Owner; multipart upload
POST <code>/api/media/courses/{courseId}/image,</code> <code>/api/media/lessons/{lessonId}/video</code>	ADMIN; multipart content media
<code>/api/admin/**</code>	ADMIN list/create/update/delete contracts
GET <code>/livez,</code> <code>/readyz</code>	Public main-listener liveness/readiness
GET <code>/actuator/health,</code> <code>/actuator/prometheus</code>	Separate Actuator filter policy; internal management listener in monitoring profile

Path parameter spelling and request binding are part of the existing contract. For example, the lesson list uses `courseID`, while progress uses `courseId`. They should be normalized only through a reviewed compatibility change with Android API interfaces and HTTP contract tests updated together.

B.2 Implementation entry points

The following source classes let a reviewer trace the major engineering arguments. Package names are relative to `backend/src/main/java/com/eduflex/`; Android classes are relative to `app/src/main/java/com/eduflex/android/`.

Table B.3: Source entry points for report claims

Concern	Classes or files to inspect
Authorization	config/SecurityConfig.java; security/AuthenticatedUser.java; route controllers
Progress	service/lesson/SaveLessonProgressUseCase.java; repository/lesson/LessonProgressRepository.java
Serialization	repository/gamification/ GamificationStatsRepository.java
Assessment	service/quiz/SubmitQuizUseCase.java; SubmitFillBlankUseCase.java in the same package; repository/quiz/QuizRepository.java
Quests	controller/gamification/ DailyQuestController.java; service/gamification/ UpdateDailyQuestProgressUseCase.java
Recovery	service/user/ForgotPasswordUseCase.java; password reset repository; migration 1.2.7
Messaging	config/ContentEventPublisher.java; config/RabbitMQConfig.java; content embedding listener
AI	service/AiCourseService.java; repository/course/CourseRepository.java
Telemetry	monitoring/EduFlexMetrics.java; monitoring rules and Grafana provisioning
Android sessions	auth/TokenManager.java; api/TokenRefreshCoordinator.java; Retrofit API interfaces
Study-time signal	ui/lesson/LessonStudyFragment.java; daily quest API interface
Correctness tests	backend/src/test/java/com/eduflex/service/ ProgressConcurrencyIntegrationTest.java from repository root

Appendix C

Configuration Reference

Variable	Purpose	Scope
API_BASE_URL	Android REST endpoint, including trailing slash	Mobile
SPRING_DATASOURCE_URL	JDBC connection for direct backend execution	Backend
SPRING_DATASOURCE_USERNAME	Application database user	Backend
SPRING_DATASOURCE_PASSWORD	Application database password	Backend
JWT_SECRETKEY	HMAC signing secret for access and refresh tokens	Backend
REDIS_PASSWORD	Redis authentication	Infra
RABBITMQ_USERNAME	Broker login name	Infra
RABBITMQ_PASSWORD	Broker login password	Infra
EDUFLEX_TIME_ZONE	Calendar zone for streaks and daily rewards	Backend
GEMINI_API_KEY	Optional server-side generation credential	Optional
GEMINI_MODEL	Optional generation model selection	Optional
SUPABASE_URL	Optional media-storage project URL	Optional
SUPABASE_SECRET_KEY	Optional server-side storage credential	Optional
GRAFANA_ADMIN_PASSWORD	Local Grafana administrator password	Monitor
POSTGRES_MONITOR_PASSWORD	Dedicated exporter role password	Monitor
EDUFLEX_ENVIRONMENT	Stable environment label on metrics	Monitor
PAYMENT_SIMULATION_ENABLED	Enable local checkout simulation; direct execution defaults off	Backend

Variable	Purpose	Scope
PASSWORD_RESET_DELIVERY_ENABLED	Activate configured SMTP recovery delivery	Backend
PASSWORD_RESET_TTL_MINUTES	Reset-code lifetime; default fifteen minutes	Backend
PASSWORD_RESET_FROM	SMTP sender for recovery messages	Backend
SMTP_HOST, SMTP_PORT	Recovery mail-server address	Backend
SMTP_USERNAME, SMTP_PASSWORD	Recovery delivery credentials	Backend
DB_POOL_MAX_SIZE	Maximum active database connections; default ten	Backend

Real values belong in untracked environment files or a deployment secret store. Provider keys and passwords must not appear in screenshots, source control, metric labels, build logs, or the submitted report archive.

Appendix D

Diagram and Evidence Index

View	Repository evidence represented
System context	Android application, backend, optional Supabase, Gemini, and Firebase integrations.
Responsibility boundaries	PostgreSQL authority, derived caches/vectors, and independent observation.
Runtime architecture	Compose services, API dependency configuration, and embedding listener.
Backend layers	Controller, service, repository, security, configuration, and monitoring packages.
Android navigation	Learner and administrator navigation XML plus screen classes.
Deployment topology	Base and monitoring Compose files, exposed ports, and named volumes.
Database model	Flyway migrations, generated records, entities, and foreign keys.
Detailed learning model	Natural-key constraints and cardinality for enrolment, progress, and attempts.
Assessment model	Question/option definitions, lesson hierarchy, and the limits of stored attempt history.
Reward model	Learner statistics, badge definitions/awards, and date-keyed quest progress.
API boundaries	Controller mappings and Spring Security route rules.
Authentication sequence	Login use case, JWT utility, refresh service, and Redis revocation.

View	Repository evidence represented
Coordinated refresh	Android TokenRefreshCoordinator and OkHttp authenticator.
Screen states	Conceptual loading/content/empty/failure acceptance model, not complete device evidence.
Password recovery	Random token, digest storage, consume lock, BCrypt update, optional SMTP.
Payment simulation	Simulation flag, owner check, partial unique index, transactional enrolment.
Learner journey	Course, payment, enrolment, lesson, quiz, progress, and reward services.
Progress state	Conditional progress transitions and concurrency integration tests.
Concurrent transactions	Two requests serialized on one learner statistics row.
Quiz workflows	Multiple-choice transaction compared with fill-blank feedback-only use case.
Cache flow	Redis configuration plus cacheable and eviction annotations.
Embedding sequence	After-commit publisher, RabbitMQ binding, listener, and embedding service.
AI retrieval	MiniLM embedding service, pgvector search, Gemini client, and fallback test.
CI flow	GitHub Actions backend, monitoring, and Android jobs.
Monitoring flow	Prometheus jobs and rules, exporters, Grafana provisioning, and Alertmanager.
Incident decision	Main-listener versus management-port investigation from the runbook.
Restoration plan	Proposed isolated restoration and state reconciliation, not an executed backup drill.

This index makes each visual auditable. The figures intentionally simplify class and column detail, but their relationships follow the repository rather than a hypothetical target architecture.

Appendix E

Submission Checklist

Replace title-page placeholders, use any declaration required by the institution, and insert real screenshots. Confirm page-count and formatting rules, and update verification dates after new test runs. Never include secrets, private Firebase configuration, signing files, real user data, or access tokens in the report.

Bibliography

- [1] AtomicJar. Testcontainers for java. <https://java.testcontainers.org/>, 2026. Accessed 25 September 2026.
- [2] Broadcom. Rabbitmq documentation. <https://www.rabbitmq.com/docs>, 2026. Accessed 25 September 2026.
- [3] Broadcom. Spring boot reference documentation. <https://docs.spring.io/spring-boot/index.html>, 2026. Accessed 25 September 2026.
- [4] Data Geekery GmbH. jooq user manual. <https://www.jooq.org/doc/latest/manual/>, 2026. Accessed 25 September 2026.
- [5] Google. Gemini api release notes. <https://ai.google.dev/gemini-api/docs/changelog>. Accessed 4 October 2026; entry dated 1 June 2026 records Gemini 2.0 shutdown.
- [6] Google. Access class information and submit an assignment. <https://support.google.com/edu/classroom/answer/16642670>, 2026. Accessed 3 October 2026.
- [7] Google. Guide to app architecture. <https://developer.android.com/topic/architecture>, 2026. Accessed 25 September 2026.
- [8] Google. Workmanager overview. <https://developer.android.com/develop/background-work/background-tasks/persistent/getting-started>, 2026. Accessed 25 September 2026.
- [9] Grafana Labs. Grafana documentation. <https://grafana.com/docs/grafana/latest/>, 2026. Accessed 25 September 2026.
- [10] Juho Hamari, Jonna Koivisto, and Harri Sarsa. Does gamification work? a literature review of empirical studies on gamification. In *Proceedings of the 47th Hawaii International Conference on System Sciences*, pages 3025–3034, 2014.

- [11] Andrew Kane. pgvector: Vector similarity search for postgresql. <https://github.com/pgvector/pgvector>, 2026. Accessed 25 September 2026.
- [12] LangChain4j Contributors. In-process (onnx) embedding models. <https://docs.langchain4j.dev/integrations/embedding-models/in-process/>. Accessed 4 October 2026.
- [13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [14] Moodle Pty Ltd. Moodle features for students. https://docs.moodle.org/405/en/images_en/2/2a/Moodle_features_students.pdf, 2026. Accessed 3 October 2026.
- [15] PostgreSQL Global Development Group. Postgresql 16 documentation: Sql dump. <https://www.postgresql.org/docs/16/backup-dump.html>. Accessed 4 October 2026.
- [16] PostgreSQL Global Development Group. Postgresql 16: Explicit locking. <https://www.postgresql.org/docs/16/explicit-locking.html>. Accessed 4 October 2026.
- [17] PostgreSQL Global Development Group. Postgresql documentation. <https://www.postgresql.org/docs/>, 2026. Accessed 25 September 2026.
- [18] Prometheus Authors. Prometheus documentation. <https://prometheus.io/docs/introduction/overview/>, 2026. Accessed 25 September 2026.
- [19] RabbitMQ. Consumer acknowledgements and publisher confirms. <https://www.rabbitmq.com/docs/confirms>. Accessed 4 October 2026.
- [20] Redgate. Flyway documentation. <https://documentation.red-gate.com/flyway>, 2026. Accessed 25 September 2026.
- [21] Redis. Redis documentation. <https://redis.io/docs/latest/>, 2026. Accessed 25 September 2026.
- [22] Sentence Transformers. all-minilm-l6-v2 model card. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>. Accessed 4 October 2026.

- [23] Spring. Declarative annotation-based caching. <https://docs.spring.io/spring-framework/reference/integration/cache/annotations.html>. Accessed 4 October 2026.
- [24] Spring Authors. Rolling back a declarative transaction. <https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/rolling-back.html>. Accessed 4 October 2026; cited for default unchecked-exception rollback behavior.
- [25] Square. Retrofit: A type-safe http client for android and java. <https://square.github.io/retrofit/>, 2026. Accessed 25 September 2026.
- [26] Yao-Ting Sung, Kuo-En Chang, and Tzu-Chien Liu. The effects of integrating mobile devices with teaching and learning on students' learning performance: A meta-analysis and research synthesis. *Computers & Education*, 94:252–275, 2016.